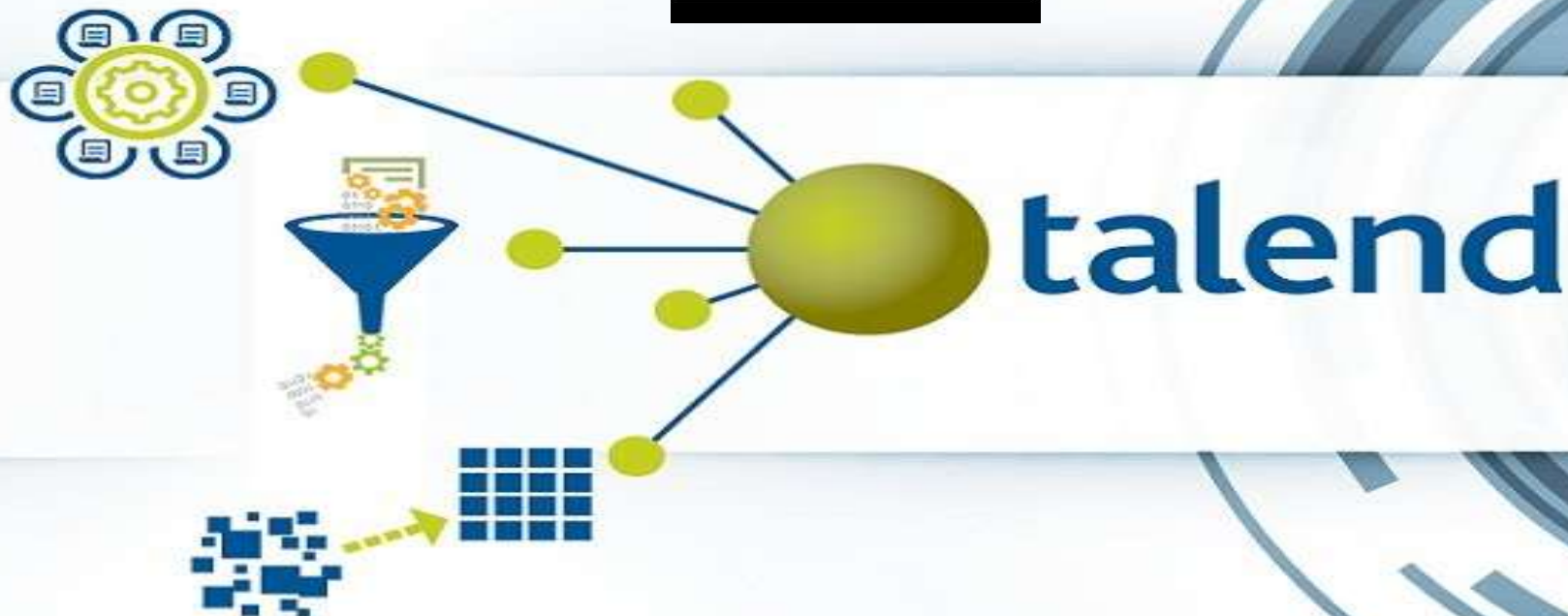
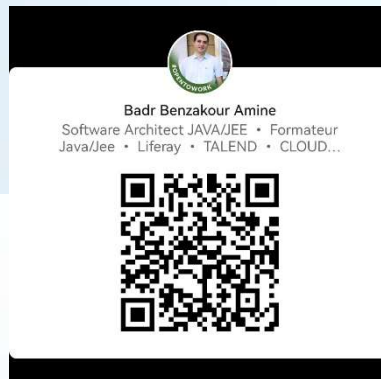


ETL, Talend Data Integration

Présenté par : Badr Benzakour Amine
Formateur et Architecte Java/Jee



Ice Breaker

Deux vérités, un mensonge



Plan

- ❖ Introduction , Définitions et cas d'utilisation
- ❖ Présentation du Talend Open Studio
- ❖ Installation et configuration du TOS
- ❖ Présentation des composants de base
- ❖ Atelier 1 : Création d'un job de transformation
- ❖ Connexions aux DBs & Transformations
- ❖ Présentation avancée des composants DBs et tMap
- ❖ Nettoyage, validation & contrôles de qualité
- ❖ Gestion des erreurs et logs
- ❖ Atelier 2 : Synchronisation de données entre deux bases PostgreSQL (DBs->tMap->CSV)
- ❖ Réutilisation, Contextes et SubJobs
- ❖ Structuration projets & Orchestration Jobs
- ❖ Atelier 3 : Structuration & Orchestration de job + création de context
- ❖ Gestion des transactions, Chargement, Performance et Debug
- ❖ Atelier 4 : Chargement massif et transactionnel d'un fichier CSV en base SQL
- ❖ Industrialisation & Déploiement d'un projet Talend
- ❖ Atelier 5 : Construire/Exporter un job en JAR exécutable + Scheduling
- ❖ Évaluation

08/11/2023

Introduction , Définitions et cas d'utilisation

Introduction & Définitions

- Dans un contexte où les entreprises génèrent et collectent chaque jour d'importants volumes de **données provenant de sources multiples**, la capacité à **intégrer, transformer et exploiter** efficacement ces informations devient un **enjeu stratégique** majeur.
- C'est dans cette perspective que les outils ETL (Extract, Transform, Load), tels que Talend, jouent un rôle essentiel.
- Un ETL permet d'automatiser les flux de données en trois grandes étapes :
 - ✓ **Extraction (Extract)** : récupération des données depuis diverses sources hétérogènes ;
 - ✓ **Transformation (Transform)** : nettoyage, enrichissement et mise en cohérence des données selon les règles métiers ;
 - ✓ **Chargement (Load)** : insertion des données préparées dans une base de destination (data warehouse, data lake, ou autre système analytique).

Introduction , Définitions et cas d'utilisation

Introduction & Définitions

- Talend est une société française fondée en 2006 : principal éditeur open source de solutions d'intégration et de gestion de données.
- Les solutions Talend sont aujourd'hui les plus utilisées et les plus déployées dans le monde (20 millions).
- Talend vous fournit un ensemble de Studios, certains open source, d'autres nécessitant une souscription
- Interface graphique d'utilisation et des centaines de composants et connecteurs intégrés, via lesquelles vous pouvez créer vos jobs et vos batchs d'un simple glisser-déposer.
- génération de code natif.
- Talend Open Studio est un logiciel qui génère un code spécifique (en Java ~~ou en Perl~~) pour chaque traitement d'intégration de données. Il est capable de travailler avec presque tous les types de fichiers...

Introduction , Définitions et cas d'utilisation

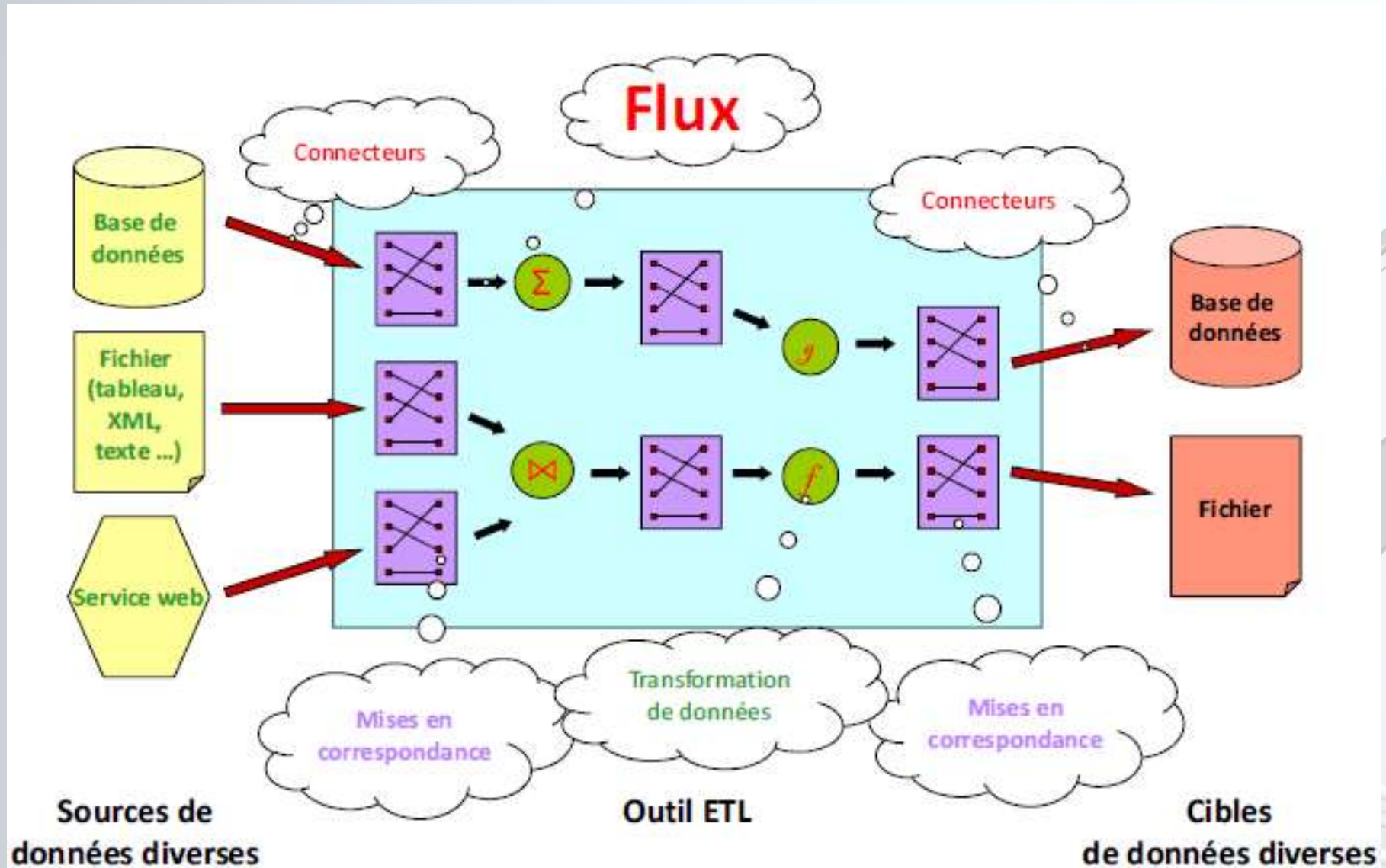
Introduction & Définitions

- Talend est « devenu **payant** » , il a été racheté par **Qlik** , la version commerciale s'intitule à présent Talend Studio et/ou Talend Data **Fabric**
- Talend a annoncé que l'édition open source de Talend Open Studio sera abandonnée à compter du 31 janvier 2024
- **La version « 8.0.1 » est la dernière version open source disponible**



Introduction , Définitions et cas d'utilisation

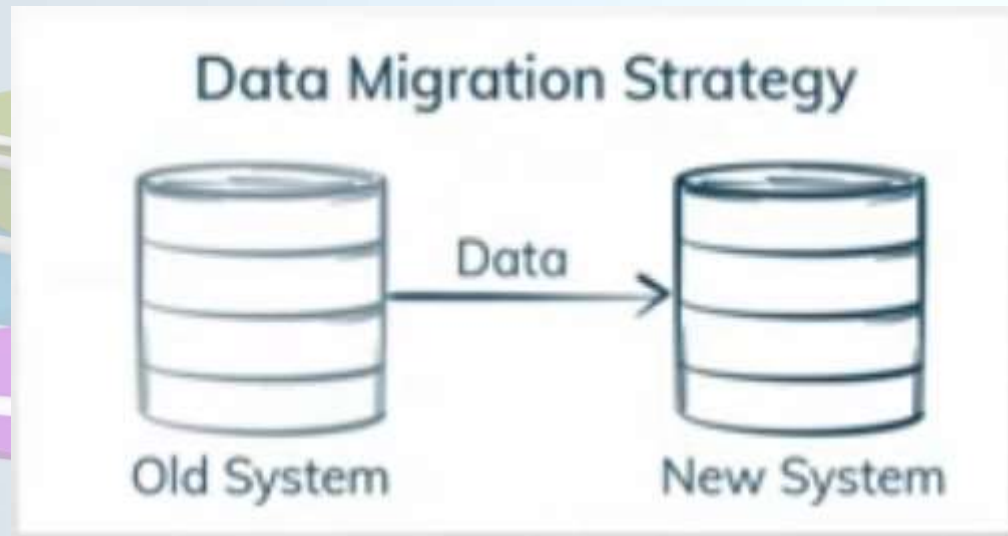
Introduction & Définitions



Introduction , Définitions et cas d'utilisation

Cas d'utilisation

La data migration (déplacer des données d'applications depuis d'anciens systèmes vers de nouveaux),
Cela exige un transfert de données entre des types ou des formats de stockages



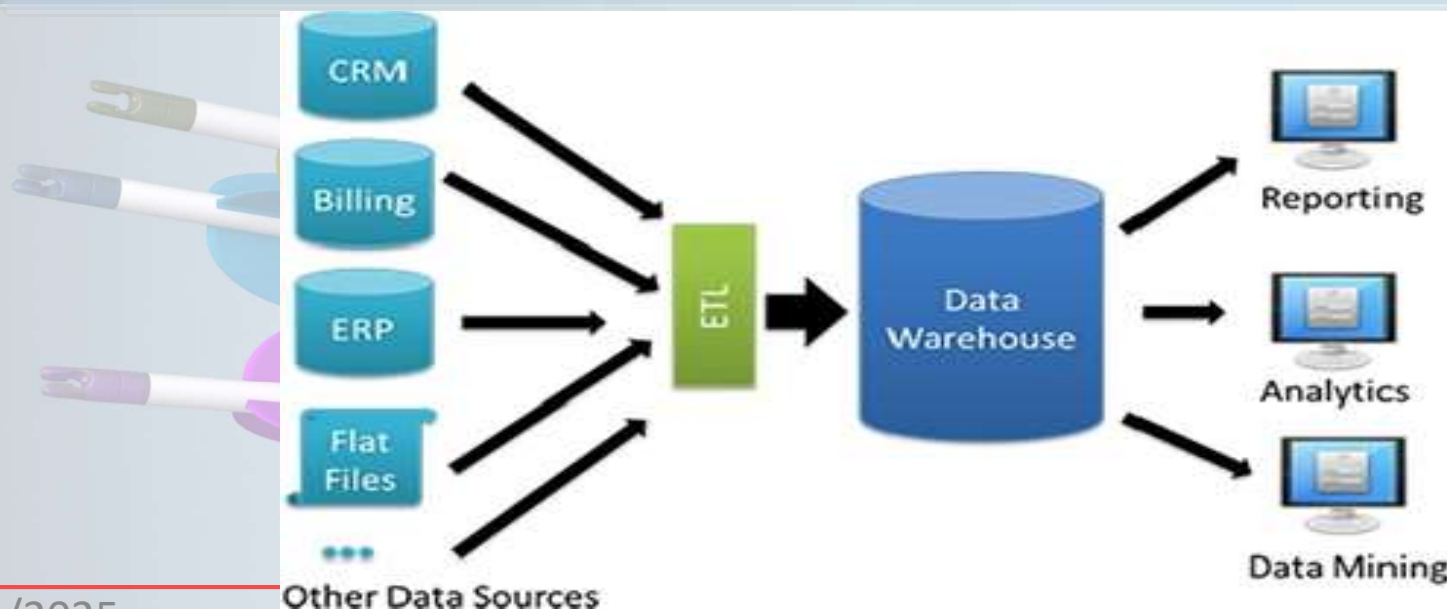
Introduction , Définitions et cas d'utilisation

Cas d'utilisation

Data Warehousing :

agréger des données transactionnelles et les stocker pour analyse et utilisation ultérieures.

Il permet de convertir les données en Business Intelligence.



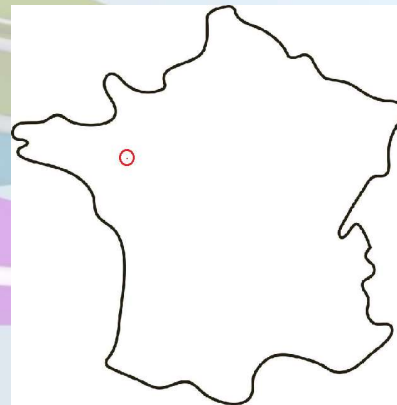
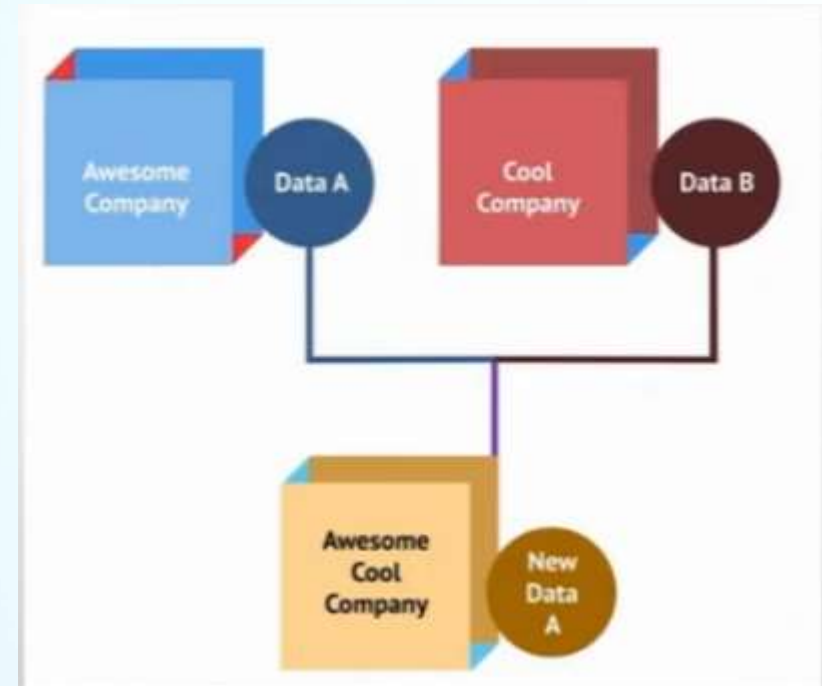
Introduction , Définitions et cas d'utilisation

Cas d'utilisation

-La Data Consolidation

Est souvent liée au déplacement de données depuis des emplacements distants vers un lieu centralisé .

-Pour éviter tout conflit ou chevauchement des données, transformer les données A dans un forma pouvant être intégré dans B



Introduction , Définitions et cas d'utilisation

Cas d'utilisation

La synchronisation de données est le processus permettant de maintenir l'exactitude des données en les copiant dans deux emplacements ou plus.

L'ajout, la modification ou la suppression d'un fichier depuis un emplacement répercutera l'action dans le nouvel emplacement.

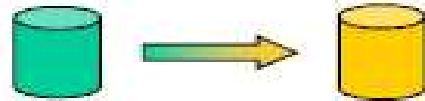


Introduction , Définitions et cas d'utilisation

Cas d'utilisation

Autres fonctions

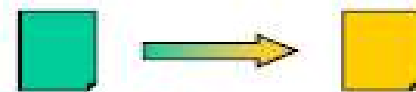
- **Changement de schéma** d'une base de données



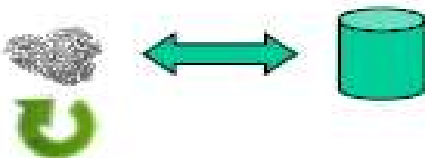
- **Vérification de la qualité** des données



- **Conversion de types** de fichiers



- **Automatisation de traitements** répétitifs



• ...

Introduction , Définitions et cas d'utilisation

Cas d'utilisation

Fonctionnalités supplémentaires possibles

- Lecture / écriture dans des **répertoires**
- Envoi / réception de **mails**
- Fonctionnalités **ELT** (Extract, Load, Transform)
- ...

Introduction , Définitions et cas d'utilisation

Cas d'utilisation

Tous ces cas d'utilisation sont implémentés et réalisés sous forme de **Jobs** et de **batches** dans la cuisine du **Talend Open Studio**

Installation et configuration Talend Open Studio

Présentation du TOS

- Talend Open Studio est à la base l'éditeur eclipse qui a été enrichi par des composants graphique d'intégration

The screenshot displays the Talend Open Studio (TOS) interface. The main window is titled "Talend Open Studio for Data Integration (8.0.1.20211109_1610) | FormationTalend_Project (Connexion: Local)". The interface includes a menu bar (Fichier, Modifier, Vue, Search, Fenêtre, Aide), a toolbar, and a sidebar on the left with a tree view showing the project structure under "LOCAL: FormationTalend_Project". The central canvas shows a job design for "OnBoardingDemoJob 0.1" with a yellow note stating "This is a simple random row generation job." and a flow diagram with a "row1 (Main)" component connected to a "tLogRow_1" component. On the right, there is a "Palette" sidebar with a search bar and a list of components categorized by type (Favoris, Récemment utilisé, Applications Métier, Bases de données, Big Data, Business Intelligence, Business, Cloud, Code personnalisé, Databases, Divers, DotNET, ELT, ESB, Fichier, Internet, Logs & Erreurs, Orchestration, Qualité de données, Système, Technique, Traitement de, Unstructured, XML).

Espace de modélisation graphique

Vous pouvez créer et disposer vos Jobs. Vous avez accès à l'onglet Designer, affichant graphiquement le Job et à l'onglet Code, vous montrant le code généré et identifiant les erreurs possibles.

FERMER PRÉCÉDENT SUIVANT

Installation et configuration Talend Open Studio

Présentation du TOS

- Talend Open Studio = éclipse + perspective d'intégration
- Génère du code Java à partir des composants/flows graphiques utilisés

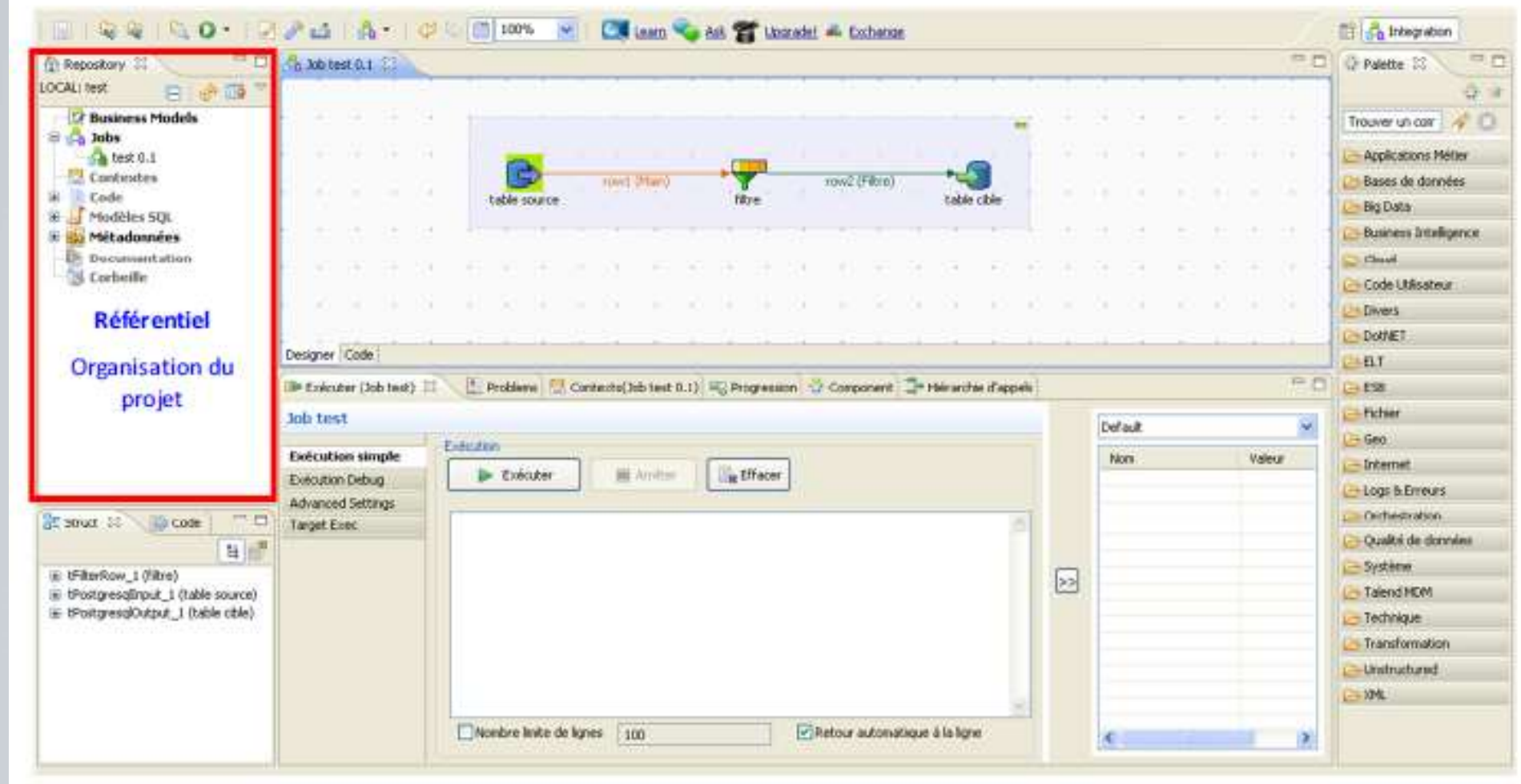
Environnement de travail de Talend



Installation et configuration Talend Open Studio

Présentation du TOS

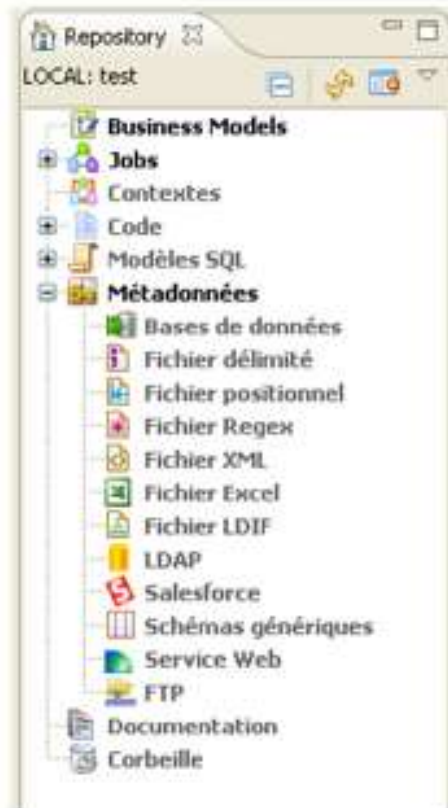
Environnement de travail de Talend



Installation et configuration Talend Open Studio

Présentation du TOS

Référentiel



– Business Models :

- Permet de stocker des schémas pouvant représenter les différentes **interactions entre les acteurs** au sein d'un système d'information, peut-être intégré à la documentation

– Jobs :

- Stocke l'ensemble des **transformations du projet**

– Code :

- Permet de voir les **routines existantes** ou d'en **créer** des nouvelles

– Contextes :

- Contient des **paramètres d'environnement** qui peuvent varier selon l'utilisation des transformations (environnement de test, production)

– Métadonnées :

- Informations relatives aux sources de données :
 - Paramètres de connexion et structures des tables pour les **bases de données**
 - Description de la **structure d'un fichier XML**
 - **Fichier Excel**
 - ...

Installation et configuration Talend Open Studio

Présentation du TOS

Environnement de travail de Talend

The screenshot displays the Talend Open Studio (TOS) interface. The main workspace shows a data job design with the following components and flow:

- Repository:** A tree view on the left showing the project structure, including 'Business Models', 'Jobs', 'test 0.1', 'Contextes', 'Code', 'Modèles SQL', 'Métadonnées', 'Documentation', and 'Corbeille'.
- Job Design:** The central canvas shows a job named 'Job test 0.1'. The flow is: 'table source' (tTableInput) → 'row1 (Filtre)' (tFilterRow) → 'row2 (Filtre)' (tFilterRow) → 'table cible' (tTableOutput).
- Designer/Code:** A tab at the bottom of the main workspace.
- Execution:** A section at the bottom left with buttons for 'Exécuter' (Execute), 'Arrêter' (Stop), and 'Effacer' (Clear). Below these are 'Exécution simple', 'Exécution Debug', 'Advanced Settings', and 'Target Exec'.
- Structure:** A tree view on the bottom left showing the job components: 'tFilterRow_1 (Filtre)', 'tPostgresInput_1 (table source)', and 'tPostgresOutput_1 (table cible)'.
- Palette:** A vertical palette on the right side, highlighted with a red box. It contains a search bar 'Trouver un composant' and a list of components categorized by type: Applications Métier, Bases de données, Big Data, Business Intelligence, Cloud, Code Utilisateur, Divers, DoNET, ETL, ESB, Fichier, Geo, Internet, Logs & Erreurs, Orchestration, Qualité de données, Système, Talend MDM, Technique, Transformation, Unstructured, and XML. Below the list is the label 'Palette' and 'Ensemble des composants utilisables'.

24

Installation et configuration Talend Open Studio

Présentation du TOS

Palette

- Permet d'accéder à tous les composants utiles à une transformation
 - Possibilité d'avoir une catégorie « favoris »
 - Composants les plus courants utilisés
 - Possibilité dans les propriétés du projet de choisir quels composants de la palette sont utilisables (permet de gagner un peu de temps au chargement)

La Palette contient différents composants techniques à utiliser pour construire vos Jobs, groupés en familles. Un composant est un connecteur préconfiguré utilisé pour effectuer une opération d'intégration de données spécifique. Il peut minimiser le code manuel requis pour utiliser des sources hétérogènes.



Installation et configuration Talend Open Studio

Présentation du TOS

Talend Open Studio for Data Integration (8.0.1.20211109_1610) | FormationTalend_Project (Connexion: Local)

Fichier Modifier Vue Search Fenêtre Aide

Onglets de configuration

Chaque onglet ouvre une vue affichant les propriétés de l'élément sélectionné dans l'espace de modélisation graphique. Ces propriétés peuvent être modifiées pour configurer des paramètres relatifs à un composant particulier ou au Job entier.

L'onglet Exécuter vous permet de lancer votre Job. Sélectionnez cet onglet et cliquez sur le bouton Exécuter pour essayer.

FERMER **PRÉCÉDENT** **SUIVANT**

row1 (Main) tLogRow_1

Job(OnBoardingDemoJob) Contexts(OnBoardingDe... Composant Exécuter (Job OnBoardin...

OnBoardingDemoJob 0.1

Principal	Nom	OnBoardingDemoJob		
Extra	Créé par :	user@talend.com	Version	0.1 M m
Stats & Logs	Création		Modification	
Version	Objectif	Used for on-boarding presentation Statut		
	Description	A simple row generation job		

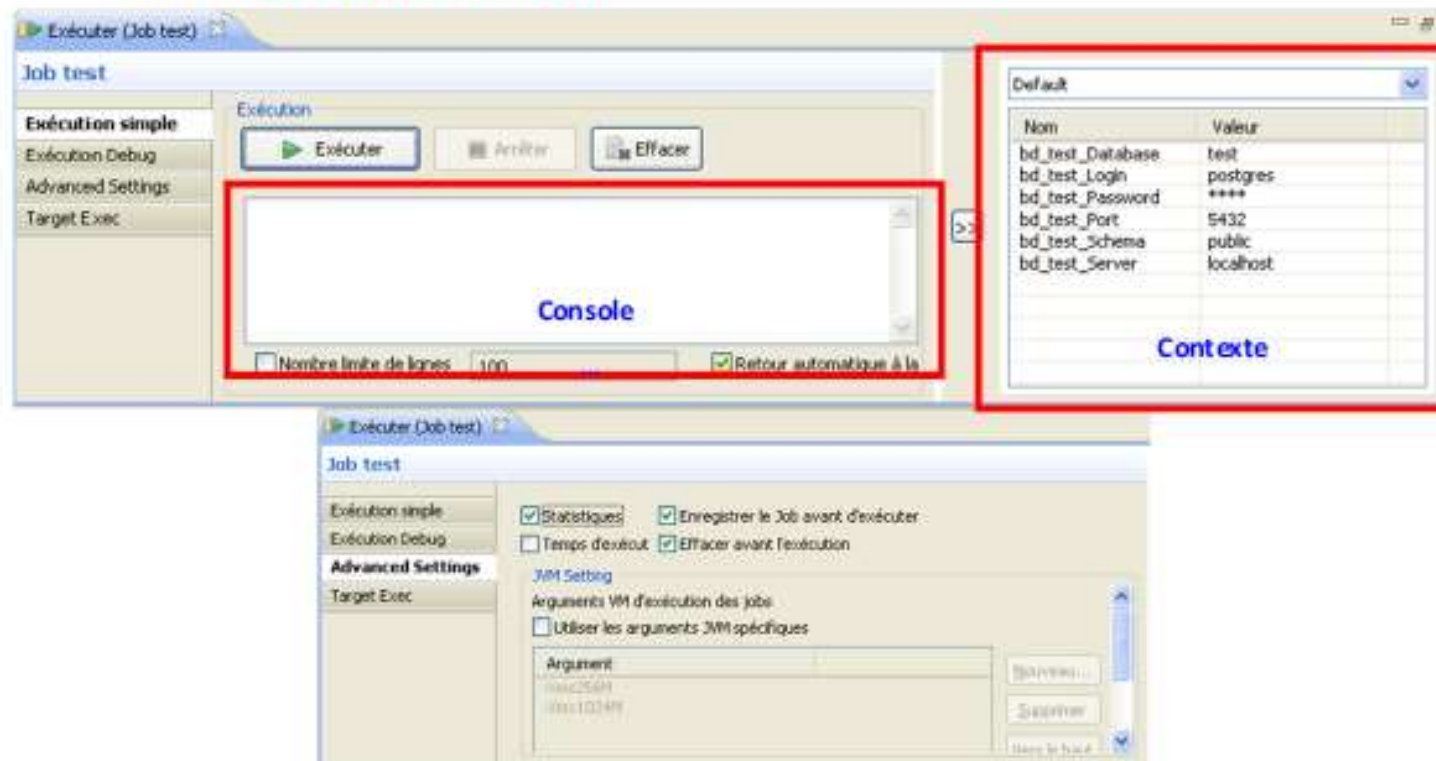
Vérifier les fonctionnalités... installer: (100%)

Installation et configuration Talend Open Studio

Présentation du TOS

Onglet Exécuter

- Permet de lancer la transformation
 - Affichage **des traces d'exécution** dans la console
 - Possibilité d'avoir des statistiques ou une trace en temps réel
 - Choix du **contexte à utiliser**



Installation et configuration Talend Open Studio

Présentation du TOS

Exécution avec test mémoire

The screenshot displays the Talend Open Studio interface during a job execution. The top section shows the job design with the following components and data:

- tRowGenerator_1**: 10 rows in 14,87s, 0,67 rows/s. Data: row1 (Main) with number 1, txt date 2006-06-26, flag false.
- AllRow**: 10 rows in 14,91s, 0,67 rows/s. Data: row3 (Main) with number 1, txt date 2006-06-26, flag false.
- tAggregateRow_1**: 9 rows in 0,03s, 272,73 rows/s. Data: row2 (Main) with number 7, txt null, flag false, count 1.
- AggregateRow**: Current row : 1.

The bottom section shows the configuration for **Job tAggregateRow_Copy**. The **Exéc. test mémoire** option is selected. The **Monitor Control** section includes an **Exécuter** button and a checkbox for **Avec un ramasse-miettes à un rythme de** (Set to Select). The **Trigger GC** section shows a memory usage graph with a scale from 200M to 1000M. The **Job execution information** section shows the start time as 16:43:30 p.m. 11/03/2025 and the end time as 16:43:44 p.m. 11/03/2025.

Installation et configuration Talend Open Studio

Téléchargement et installation du Java 11 & TOS 8

- Télécharger JDK 11 depuis le site officiel d'oracle , ou bien depuis le lien ci-dessous :

https://drive.google.com/file/d/1fd58QaO3_2FaNXBjANKlknGkdR0KsWep/view?usp=sharing

- Définir au niveau des Variables environnements JAVA_HOME & MAJ du PATH avec %JAVA_HOME%\bin

- Télécharger Talend Open Studio 8.0.1 depuis :

https://www.reddit.com/r/Talend/comments/1bkvrsu/talend_open_studio_for_data_integration_download/?tl=fr

<https://gofile.io/d/ulmHb7>

<https://drive.google.com/file/d/1vejO8Z4VA0zTqsZwHQOBxfGMTtZkexB/view?usp=sharing>

- Dézippez le fichier téléchargé à l'aide de 7-Zip par exemple (auquel cas vous devez télécharger 7-Zip).
- Exécutez le fichier de type application (.exe)

Présentation des composants de base tFileInputDelimited & tFileOutputDelimited

The screenshot displays the Talend Studio interface. In the background, a job design is visible with components **tDBConnection_1**, **tDBCommit_1**, and **tFileInputDelimited_1**. The **tFileInputDelimited_1** component is selected, and its configuration window is open in the foreground.

Schema of tFileInputDelimited_1

Column	Key	Type	☒ Nul...	Date Patter...	Length	Precisi...	Def...	Com...
Id	☐	Integer	☒		10			
CountryName	☐	String	☒		100			
Sale	☐	String	☒		10			

tutorialgateway.org

Basic settings

- Property Type: Built-In
- Schema: Built-In **Edit schema** (highlighted with a red box)
- File name/Stream: "C:/Users/sures/Downloads/Input Files/Fuzzy_Source.txt" *
- Row Separator: "\n" Field Separator: "," *
- ☐ CSV options
- Header: 1 Footer: 0 Limit:
- ☒ Skip empty rows ☐ Uncompress as zip file ☐ Die on error

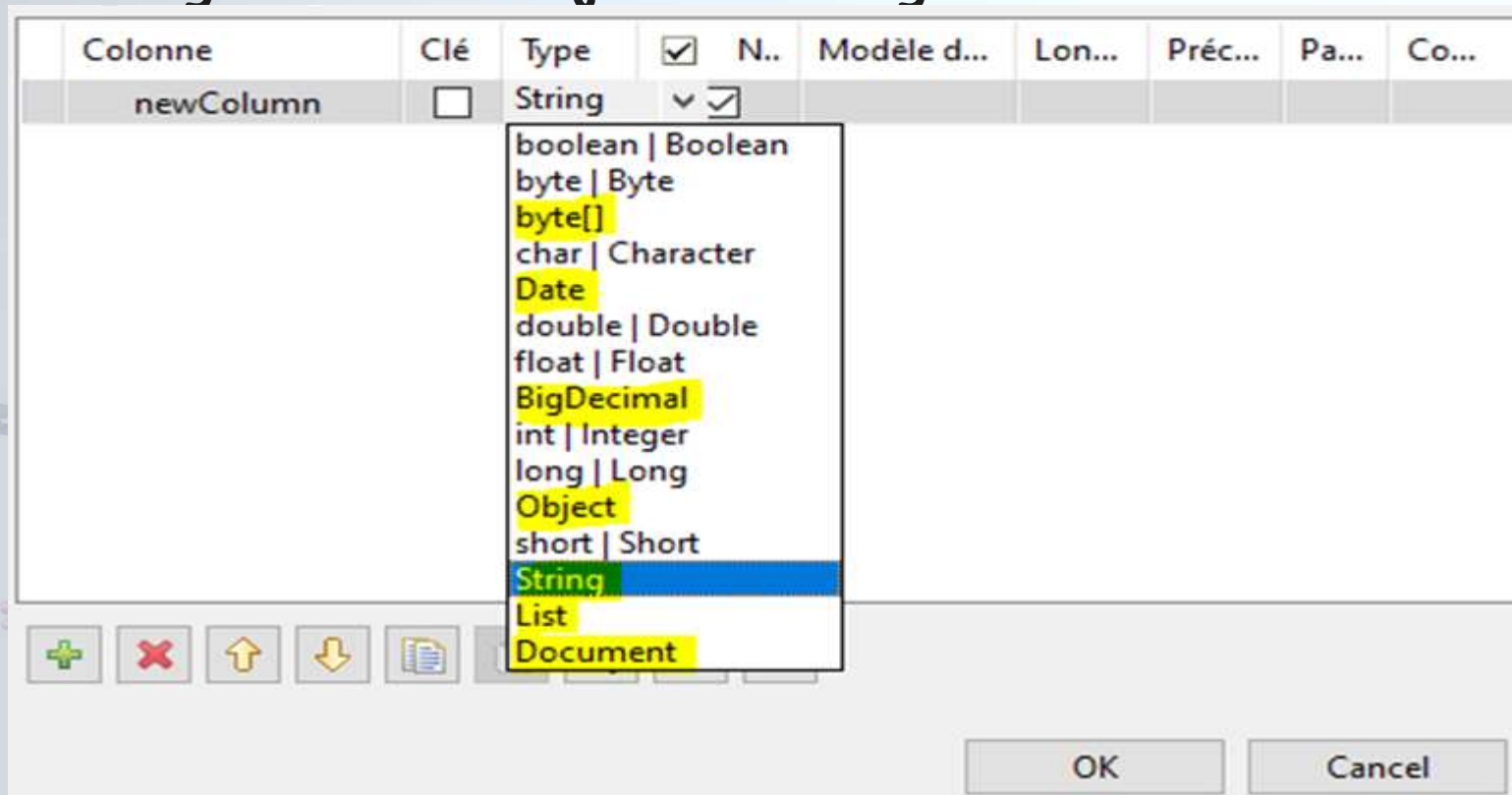
Présentation des composants de base

Schéma & type de données en Talend

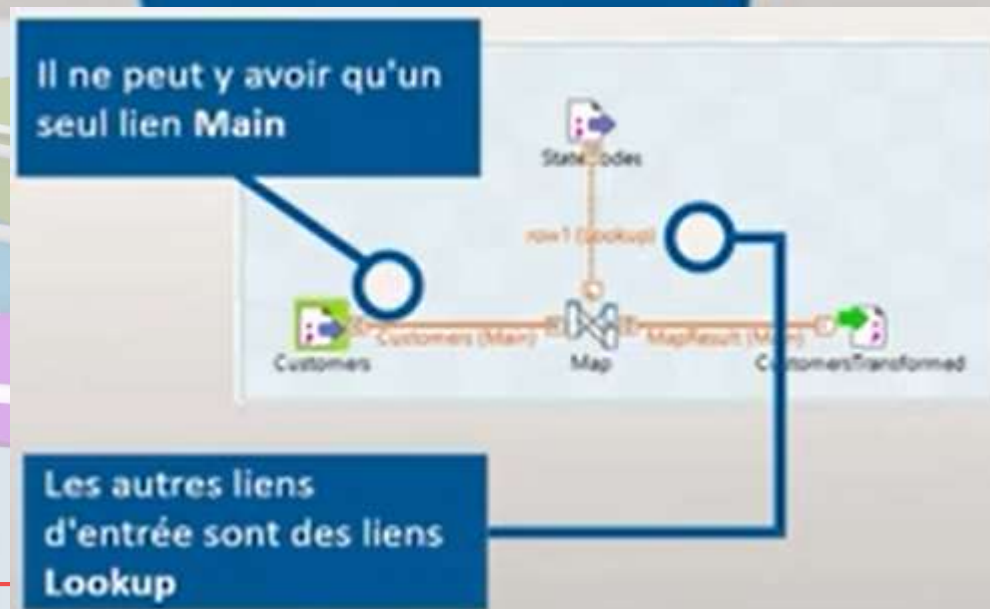
- Dans Talend les données en input/output peuvent être de :

15 types

8 types primitifs de Java et leurs enveloppes + byte[] + Date + BigDecimal + Object + String + List + Document

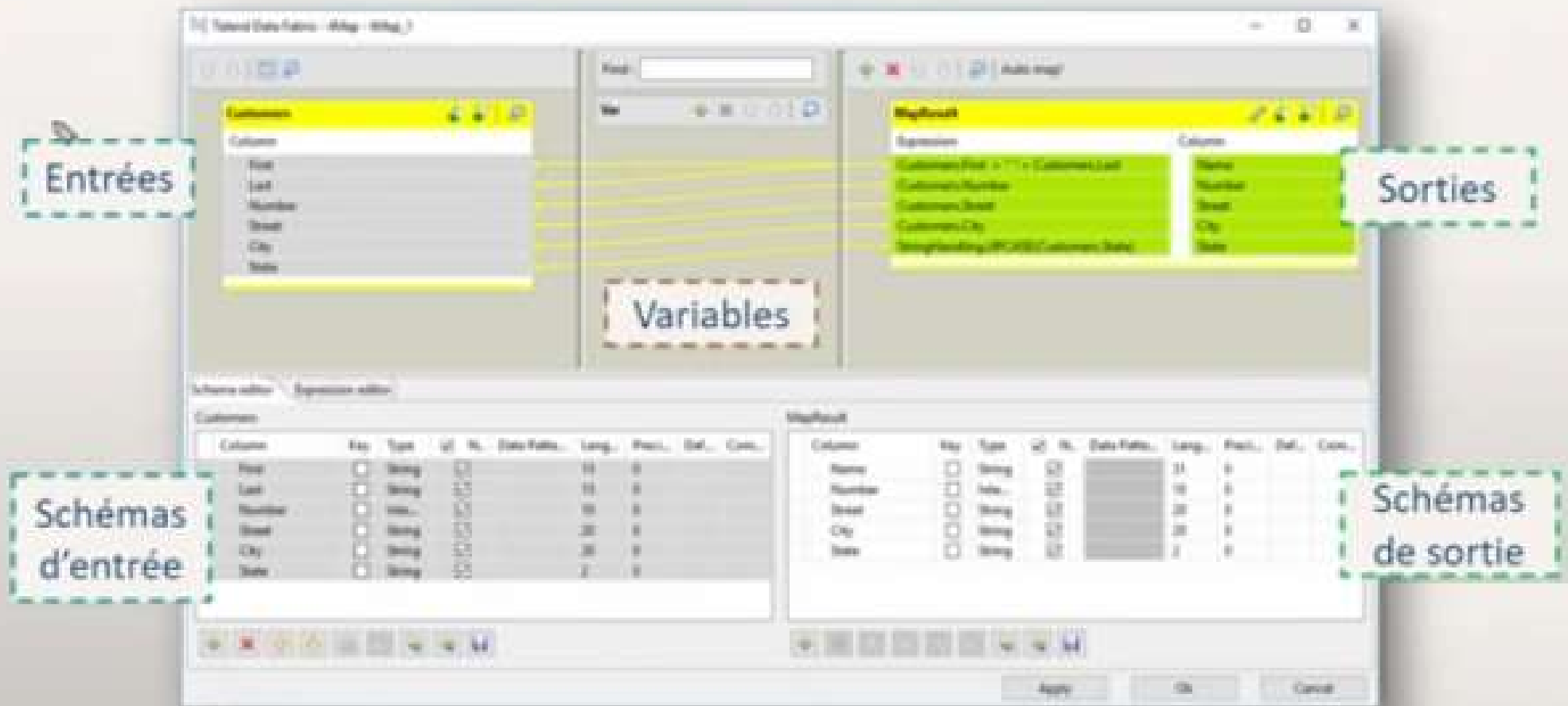


Présentation des composants de base tMap



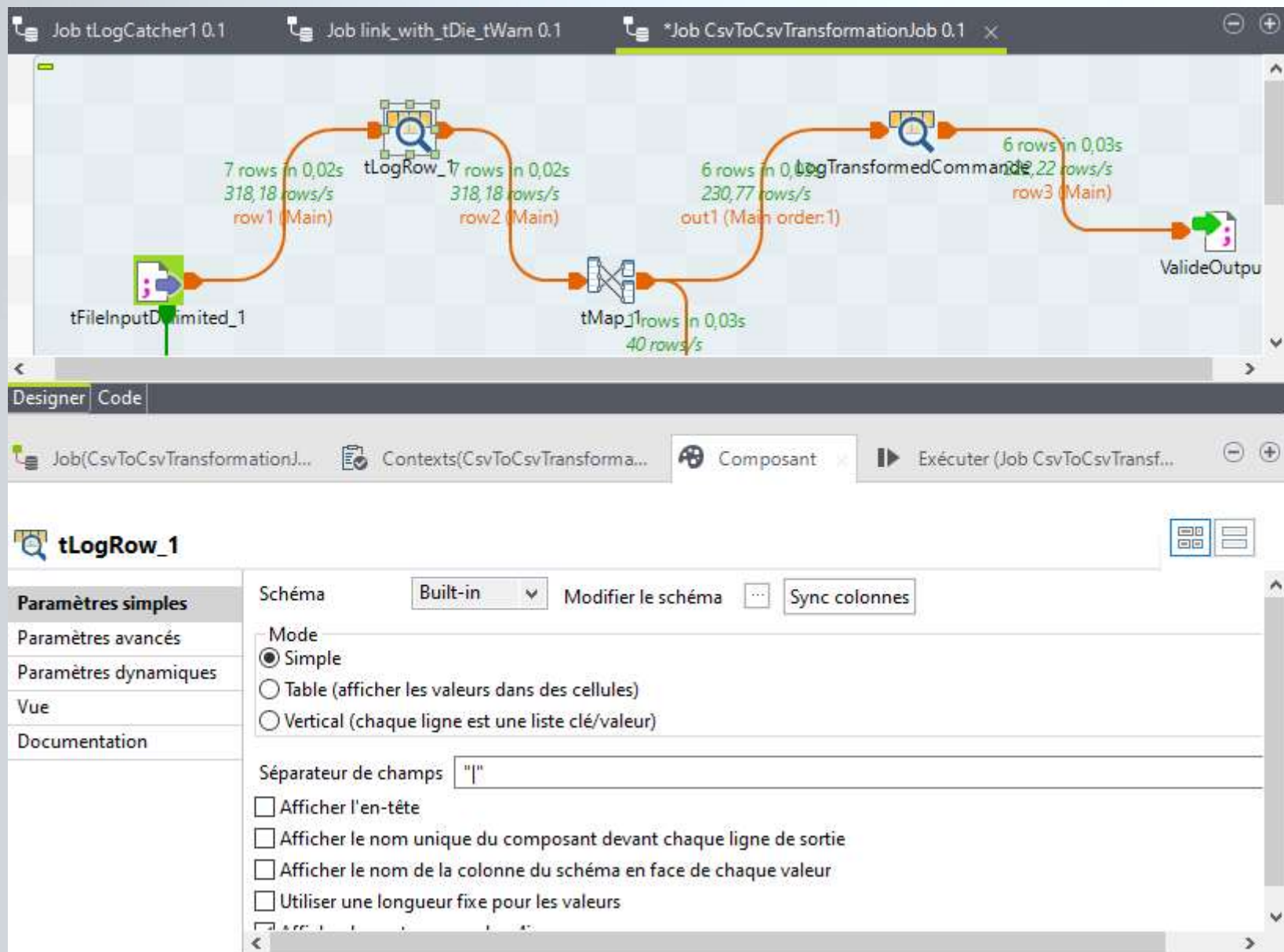
Présentation des composants de base tMap

Disposition des panneaux dans l'éditeur du tMap :



Présentation des composants de base

tLogRow



Présentation des composants de base

tLogRow

Exécution



Exécuter



Arrêter



Effacer

Démarrage du Job CsvToCsvTransformationJob à 15:53 01/11/2025.

[statistics] connecting to socket on port 3568

[statistics] connected

[WARN] 15:54:00 formationtalend_project.csvtocsvtransformationjob_0_1.CsvToCsvTransformationJob-

tLogRow_1							
order_id	customer_name	product	quantity	unit_price	date	city	country
1001	John SMITH	Laptop	2	700.0	16-06-0007	paris	france
1002	Amine Badr	Mouse	10	15.0	15-06-0008	casablanca	maroc
1003	Sarah DOE	Laptop	1	800.0	15-06-0009	Paris	France
1004	Incomplete		0	null	16-06-0011	rabat	maroc
1005	John SMITH	Keyboard	3	30.0	15-06-0012	paris	france
1006	Marie CURIE	Laptop	1	750.0	15-06-0013	lyon	france
1007	Ali Hassan	Mouse	5	12.0	15-06-0013	tanger	maroc

LogTransformedCommande								
order_id	customer_name	product	quantity	unit_price	date	city	country	totalPrice
1001	JOHN SMITH	Laptop	2	700.0	16-06-0007	paris	FRANCE	1400.0
1002	AMINE BADR	Mouse	10	15.0	15-06-0008	casablanca	MAROC	150.0
1003	SARAH DOE	Laptop	1	800.0	15-06-0009	Paris	FRANCE	800.0
1005	JOHN SMITH	Keyboard	3	30.0	15-06-0012	paris	FRANCE	90.0
1006	MARIE CURIE	Laptop	1	750.0	15-06-0013	lyon	FRANCE	750.0
1007	ALI HASSAN	Mouse	5	12.0	15-06-0013	tanger	MAROC	60.0

Présentation des composants de base

tSortRow

The screenshot shows the configuration interface for the **tSortRow** component. The component is highlighted with a yellow circle. A red circle highlights the **Installer...** button. A dialog box is open showing the list of modules not installed for the component.

Paramètres simples

Paramètres avancés

Paramètres dynamiques

Vue

Documentation

Schema Built-in **Modifier le schéma** **Sync colonnes**

Critères

Colonne du schéma	tri num ou alpha ?	Ordre asc ou desc ?
totalPrice	num	DESC

Le composant tSortRow requiert les modules tiers suivants :

Liste des modules non installés pour le composant tSortRow

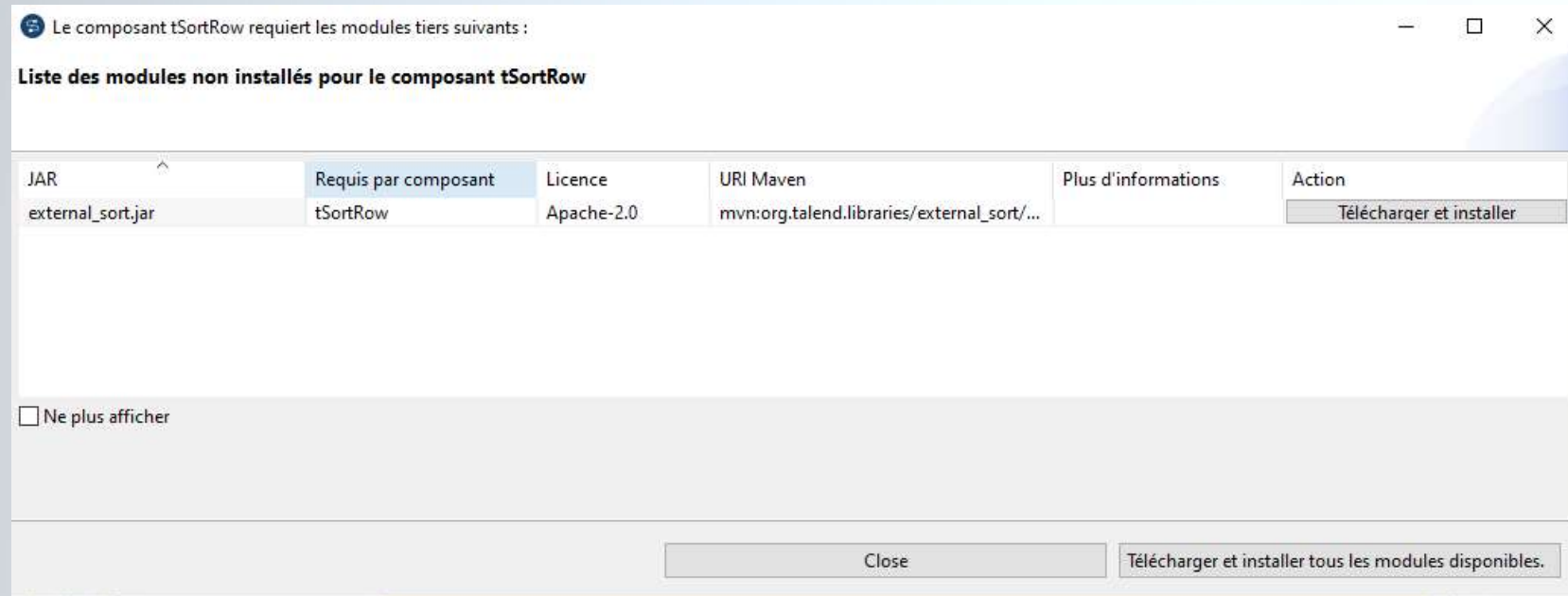
JAR	Requis par composant	Licence	URI Maven	Plus d'informations	Action
external_sort.jar	tSortRow	Apache-2.0	mvn:org.talend.libraries/external_sort/...		Télécharger et installer

☐ Ne plus afficher

Close Télécharger et installer tous les modules disponibles.

Présentation des composants de base

tSortRow



LogTransformedCommande								
order_id	customer_name	product	quantity	unit_price	date	city	country	totalPrice
1001	JOHN SMITH	Laptop	2	700.0	16-06-0007	paris	FRANCE	1400.0
1003	SARAH DOE	Laptop	1	800.0	15-06-0009	Paris	FRANCE	800.0
1006	MARIE CURIE	Laptop	1	750.0	15-06-0013	lyon	FRANCE	750.0
1002	AMINE BADR	Mouse	10	15.0	15-06-0008	casablanca	MAROC	150.0
1005	JOHN SMITH	Keyboard	3	30.0	15-06-0012	paris	FRANCE	90.0
1007	ALI HASSAN	Mouse	5	12.0	15-06-0013	tanger	MAROC	60.0

Atelier 1 : Création d'un job de transformation

- En utilisant les composants `tFileInputDelimited`, `tMap`, `tLogRow`, `tFileOutputDelimited`, `tSortRow...`, construit un job qui parcourt un fichiers de commandes clients CSV

(`order_id;customer_name;product;quantity;unit_price;date;city;country`)

- 1) Logger les lignes traités
- 2) Filtrer les lignes erronées (product et/ou quantity vides) les logger, leur ajouter le motif de rejet et les mettre dans un fichier de rejets avec `tFileOutputDelimited ~/orders_rejects.csv`
- 3) Pour les commandes valides, transformer les données `customer_name` & `country` en majuscule.
- 4) Supprimer l'info de la ville de la commande
- 5) Calculer le montant globale de la commande et l'ajouter à la sortie du flux .
- 6) Trier les résultats par montant de commande
- 7) Enregistre le résultat dans un autre fichier CSV structuré.
- 8) Paramétrage dynamique via contextes des infos du fichier csv en entrée,

Connexions aux DBs & Transformations

tDBInput

Designer Code

Job(SynchronizationDBsJob 0.1) Contexts(SynchronizationDBsJob) Composant Exécuter (Job SynchronizationDBsJob)

produit_backup(tDBInput_2)(PostgreSQL)

Paramètres simples

Paramètres avancés

Paramètres dynamiques

Vue

Documentation

Database: PostgreSQL Appliquer

Type de propriété: Référentiel Bases de données (POSTGRESQL):DB_BACKUP

☐ Utiliser une connexion existante

Version de la base de données: V9 et plus

Hôte: localhost Port: 5432

Base de données: DB_BACKUP Schéma: public

Utilisateur: postgres Mot de passe: *****

Schéma: Référentiel Bases de données (POSTGRESQL):DB_BACKUP Modifier le schéma

Nom de la table: produit

Type de requête: Built-in Guess Query Guess schema

Requête: "SELECT
\"public\".\"produit\".\"id_produit\",
\"public\".\"produit\".\"code_produit\",

Connexions aux DBs & Transformations

tDBOutput

The screenshot displays the Talend Studio interface during a job execution. At the top, a data flow is visible with two components: 'UPDATED_PRODUCT' and 'produit'. The 'UPDATED_PRODUCT' component shows '6 rows in 0,2s' and '30,61 rows/s'. The 'produit' component shows '6 rows in 0,36s' and '16,62 rows/s'. Below the flow, the 'Designer' tab is active, showing the configuration for the 'produit'(tDBOutput_1)(PostgreSQL) component.

Configuration details for 'produit'(tDBOutput_1)(PostgreSQL):

- Database:** PostgreSQL (with 'Appliquer' button)
- Type de propriété:** Référentiel (with 'Bases de données (POSTGRESQL):DB_BACKU' dropdown)
- ☐ Utiliser une connexion existante
- Version de la base de données:** V9 et plus
- Hôte:** "localhost" (with asterisk icon)
- Port:** "5432" (with asterisk icon)
- Base de données:** "DB_BACKUP" (with asterisk icon)
- Schéma:** "public" (with asterisk icon)
- Utilisateur:** "postgres" (with asterisk icon)
- Mot de passe:** "*****" (with asterisk icon)
- Table:** "produit" (with asterisk icon and 'Go' button)
- Action sur la table:** Par défaut (dropdown)
- Action sur les données:** Update (dropdown)
- Schéma:** Référentiel (dropdown) with 'Bases de données (POSTGRESQL):DB_BACKU' dropdown, 'Modifier le schéma' button, and 'Sync colonnes' button
- Source de données:** This option only applies when deploying and running in the Talend Runtime
 - ☐ Spécifier un alias de source de données
 - ☐ Arrêter en cas d'erreur

Connexions aux DBs & Transformations

tDBOutput

The screenshot displays the Talend Studio interface during the configuration of a **tDBOutput_1** component. The top workflow bar shows data flow from **UpdatedProduct (Main order:1)** through a **UPDATED_PRODUCT** transformation to the **produit** target, with performance metrics of 6 rows in 0,2s at 30,61 rows/s and 6 rows in 0,36s at 16,62 rows/s.

The **Designer** tab is active, showing the configuration for **"produit"(tDBOutput_1)(PostgreSQL)**. The **Paramètres simples** section is expanded, showing the following settings:

- Database:** PostgreSQL (with an **Appliquer** button)
- Type de propriété:** Référentiel
- Bases de données (POSTGRESQL):** DB_BACKU
- ☐ **Utiliser une connexion existante**
- Version de la base de données:** V9 et plus
- Hôte:** localhost
- Port:** 5432
- Base de données:** DB_BACKUP
- Schéma:** public
- Utilisateur:** postgres
- Mot de passe:** *****
- Table:** produit
- Action sur la table:** Par défaut
- Action sur les données:** Update (with a dropdown menu open showing options: Insert, Update, Insérer ou mettre à jour, Update ou insert, Supprimer)
- Schéma:** Référentiel
- Bases de données (POSTGRESQL):** DB_BACKU
- ☐ **Source de données** (This option only applies when deploying and running in the Talend Runtime)
- ☐ **Spécifier un alias de source de données**
- ☐ **Arrêter en cas d'erreur**

Connexions aux DBs & Transformations

tMap (jointures, filtres, expressions)

rowBackup

Column
id_produit
code_produit
nom_produit
description
categorie
marque
prix_achat
prix_vente
stock
seuil_alerte
unite_mesure
date_ajout
id_fournisseur
actif

rowPrimaire

Property	Value
Lookup Model	Charger une fois
Match Model	Correspondance unique
Join Model	Left Outer Join
Store temp data	false

Clé d'expr.

Column
rowBackup.id_produit
id_produit
code_produit
nom_produit

Var

Expression	Type	Variable
(rowPrimaire.id_produit == null)...	String	☒ statusRow

NewOrUpdatedProduct

Expression: (rowPrimaire != null && ("AJOUTE".equals (Var.statusRow) || "MODIFIE".equals (Var.statusRow))

Expression	Column
rowPrimaire.id_produit	id_produit
rowPrimaire.code_produit	code_produit
rowPrimaire.nom_produit	nom_produit
rowPrimaire.description	description
rowPrimaire.categorie	categorie
rowPrimaire.marque	marque
rowPrimaire.prix_achat	prix_achat
rowPrimaire.prix_vente	prix_vente
rowPrimaire.stock	stock
rowPrimaire.seuil_alerte	seuil_alerte
rowPrimaire.unite_mesure	unite_mesure
rowPrimaire.date_ajout	date_ajout
rowPrimaire.id_fournisseur	id_fournisseur
rowPrimaire.actif	actif
Var.statusRow	status

DeletedProduct

Expression: (rowBackup != null && "SUPPRIME".equals (Var.statusRow))

Expression	Column
rowBackup.id_produit	id_produit
rowBackup.code_produit	code_produit
rowBackup.nom_produit	nom_produit
rowBackup.description	description

Éditeur de schéma Éditeur d'expression

rowPrimaire NewOrUpdatedProduct

Appliquer Ok Annuler

Connexions aux DBs & Transformations

tMap (jointures, filtres, expressions)

The screenshot shows the configuration of a tMap transformation. The 'rowBackup' table is selected, and its columns are listed. The 'Var' panel shows an expression for filtering rows where 'id_produit' is null. The 'lookupRejectedRows' panel is active, showing properties for handling rejected rows. The 'Catch output reject' and 'Catch lookup inner join reject' options are both set to 'false'. The bottom panel shows the schema for 'rowBackup' and 'lookupRejectedRows', both with columns 'id_produit' of type Integer.

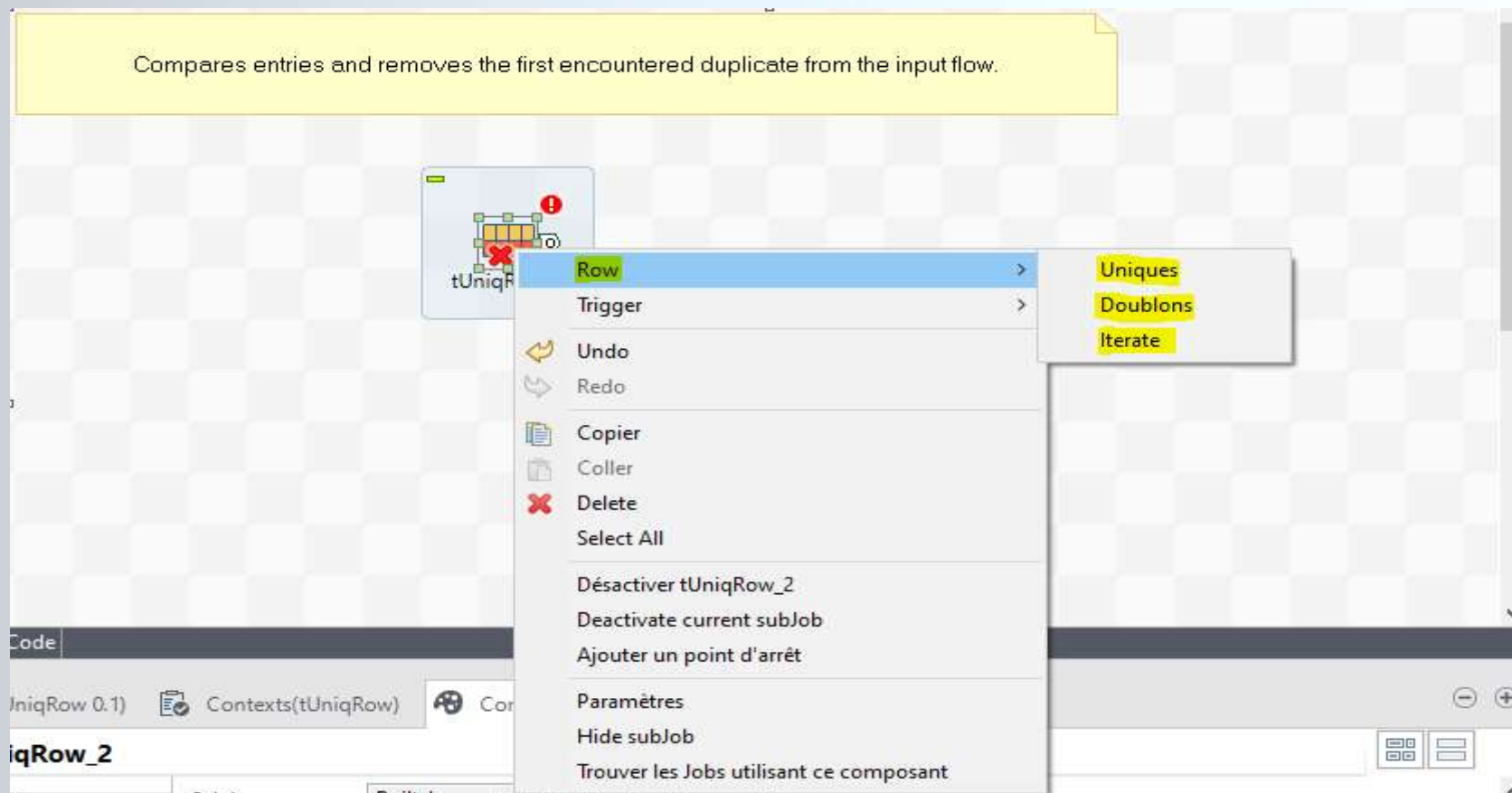
- **“Catch output reject”**, permet de récupérer les lignes rejetées à cause d’erreurs de mapping ou de filtrage dans la sortie (Output) d’un tMap
- L’option **“Catch lookup inner join reject”** ne fonctionne **que** lorsque la jointure entre le flux **Main** et le **Lookup** est de type **Inner Join**, et permet de récupérer les lignes du **Main** sans correspondance

Nettoyage, validation & contrôles de qualité tUniqRow

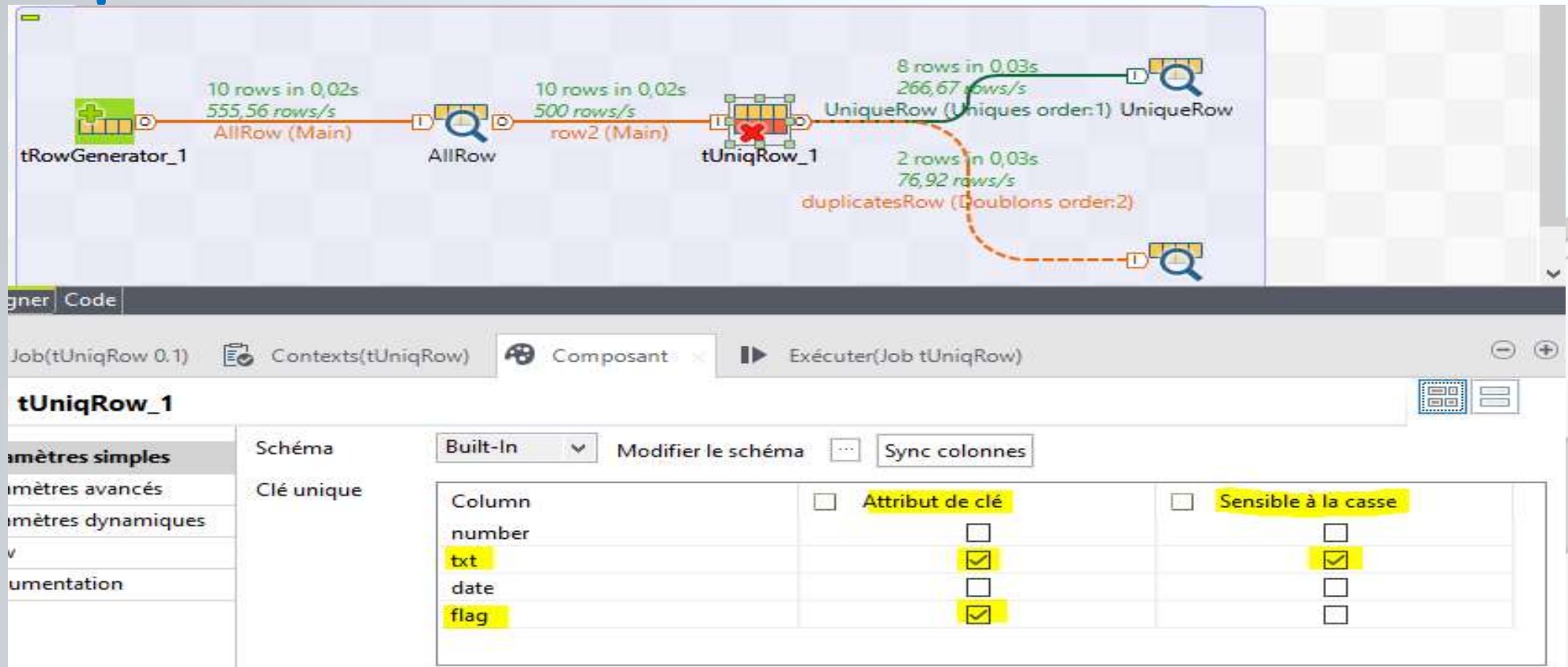


➤ Le composant tUniqRow :

- ✓ Permet d'éliminer les doublons dans un flux de données



Nettoyage, validation & contrôles de qualité tUniqRow



AllRow			
number	txt	date	flag
8	text6	2006-06-26	true
3	text4	2006-04-24	false
4	text8	2005-08-28	true
5	text4	2006-05-25	false
1	text9	2005-07-27	true
7	text7	2006-05-25	true
8	text2	2006-04-24	true
1	text8	2005-07-27	true
9	text3	2004-11-21	false
9	text4	2006-04-24	false

UniqueRow			
number	txt	date	flag
8	text6	2006-06-26	true
3	text4	2006-04-24	false
4	text8	2005-08-28	true
1	text9	2005-07-27	true
7	text7	2006-05-25	true
8	text2	2006-04-24	true
9	text3	2004-11-21	false

DuplicateRow			
number	txt	date	flag
5	text4	2006-05-25	false
1	text8	2005-07-27	true
9	text4	2006-04-24	false

Nettoyage, validation & contrôles de qualité

tReplace



➤ Le composant tReplace :

- ✓ Permet de remplacer toute occurrence d'une valeur par son substituant dans un flux de données
- ✓ Permet de rechercher par regex

Carries out a Search & Replace operation in the input columns defined. Helps to cleanse all files before further processing.

10 rows in 0,03s
322,58 rows/s
row1 (Main)

10 rows in 0,03s
322,58 rows/s
row3 (Main)

10 rows in 0,05s
196,08 rows/s
row2 (Main)

tRowGenerator_1 → AllRow → tReplace_1 → AllRowAfterReplace

AllRowAfterReplace			
number	txt	date	flag
6	newValue10	2005-07-27	true
9	newValue1	2005-08-28	false
6	newValue7	2006-06-26	true
8	newValue1	2007-02-22	false
3	newValue1	2007-01-21	true
1	newValue6	2005-08-29	false
10	newValue2	2005-08-29	true
5	newValue7	2005-08-28	true
6	newValue4	2006-06-26	false
1	newValue5	2005-08-28	false

Designer Code

Job(tReplace_Copy 0.1) Contexts(tReplace_Copy) Composant Exécuter (Job tReplace_Copy)

tReplace_1

Paramètres simples Paramètres avancés

Schéma Built-in Modifier le schéma Sync colonnes

☒ Mode simple Simple mode works with regex expression(in last version)

Colonne d'entr...	Rechercher	Remplacer par	☒ Tout le mot	☐ Sensible ...	☐ Expressio...	Com
txt	"text"	"newValue"	☐	☐	☐	

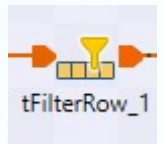
chercher/Remplacer

AllRow			
number	txt	date	flag
6	text10	2005-07-27	true
9	text1	2005-08-28	false
6	text7	2006-06-26	true
8	text1	2007-02-22	false
3	text1	2007-01-21	true
1	text6	2005-08-29	false
10	text2	2005-08-29	true
5	text7	2005-08-28	true
6	text4	2006-06-26	false
1	text5	2005-08-28	false

☐ Mode avancé (recherche avec modèle de regex)

Nettoyage, validation & contrôles de qualité

tFilterRow



- Le composant tFilterRow, est un outil de filtrage conditionnel :
 - ✓ Permet de laisser passer que les lignes qui respectent les conditions définies (Sortie : Filter)
 - ✓ Sortie : Reject → Les lignes qui ne respectent pas les conditions

Designer Code

Job(tFilterRows_Copy 0.1) Contexts(tFilterRows_Copy) Composant Exécuter (Job tFilterRows_Copy)

tFilterRow_1

Paramètres simples

Paramètres avancés

Paramètres dynamiques

Vue

Documentation

Schéma Built-in Modifier le schéma Sync colonnes

Opérateur logique utilisé pour combiner les conditions Et *

Conditions

Colonne d'entrée	Fonction	Opérateur	Valeur
number	Vider	Égal à	5
txt	Longueur	Égal à	5

Utiliser le mode avancé

FILTRED_ROW			
number	txt	date	flag
5	text1	2005-08-29	false
5	text2	2005-08-28	true

REJECTED_ROW			
number	txt	date	flag
1	text1	2006-06-26	true
1	text2	2005-07-27	true
1	text2	2007-02-22	false
1	5text5	2005-08-29	false
5	4text4	2006-06-26	false
5	3text3	2007-01-21	false
1	text2	2006-06-26	false
5	5text5	2007-02-22	true

Nettoyage, validation & contrôles de qualité

Agrégations : tAggregateRow



➤ le composant tAggregateRow sert dans le cas suivant :

- ✓ tu veux regrouper et calculer des agrégats (sommes, moyennes, etc.), un peu comme une clause GROUP BY en SQL.

tAggregateRow_1

Paramètres simples

Paramètres avancés

Paramètres dynamiques

View

Documentation

Schéma

Built-In

Modifier la fonction

Group by

Colonne de sortie

number

txt

Opérations

Colonne de sortie

count

Fonction

count

Position de la colonne d'entrée

number

txt

Position de la colonne d'entrée

number

☐ Ignorer les valeurs ...

AllRow			
number	txt	date	flag
7	text5	2005-08-29	true
1	text4	2005-08-28	true
7	text5	2007-02-22	true
7	text1	2005-08-28	false
7	text3	2007-03-23	true
7	text2	2007-03-23	true
1	text3	2007-02-22	true
10	text5	2005-07-27	false
1	text2	2007-03-23	true
7	text1	2006-04-24	true

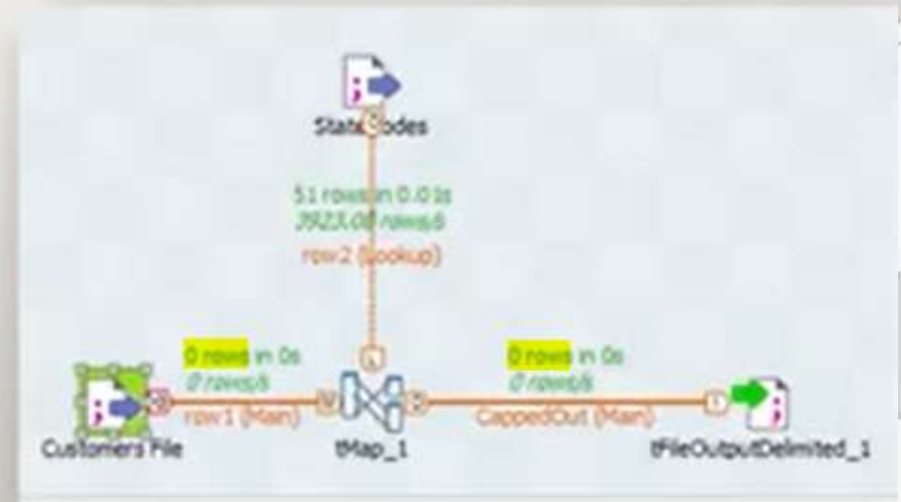
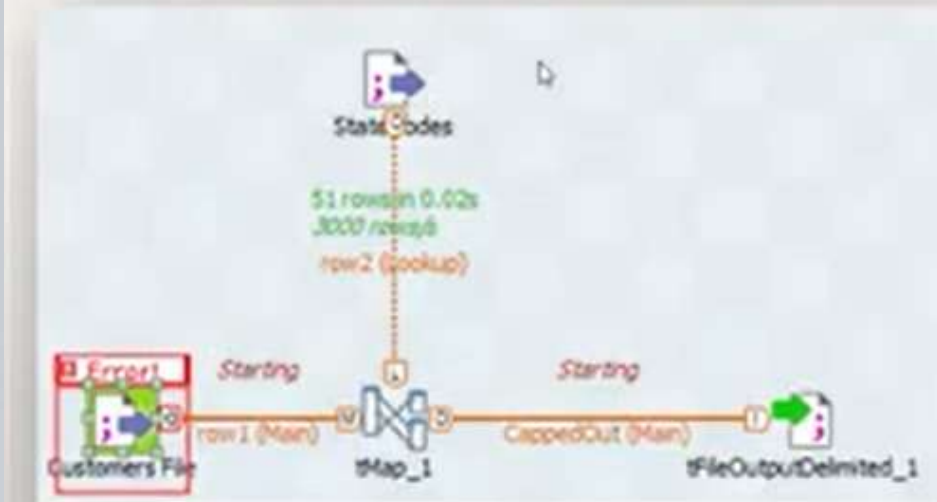
AggregateRow				
number	txt	date	flag	count
7	text5	null	false	2
7	text3	null	false	1
1	text3	null	false	1
1	text4	null	false	1
1	text2	null	false	1
7	text1	null	false	2
7	text2	null	false	1
10	text5	null	false	1

Connexions aux DBs & Transformations

Gestion des erreurs

Si un composant rencontre une erreur

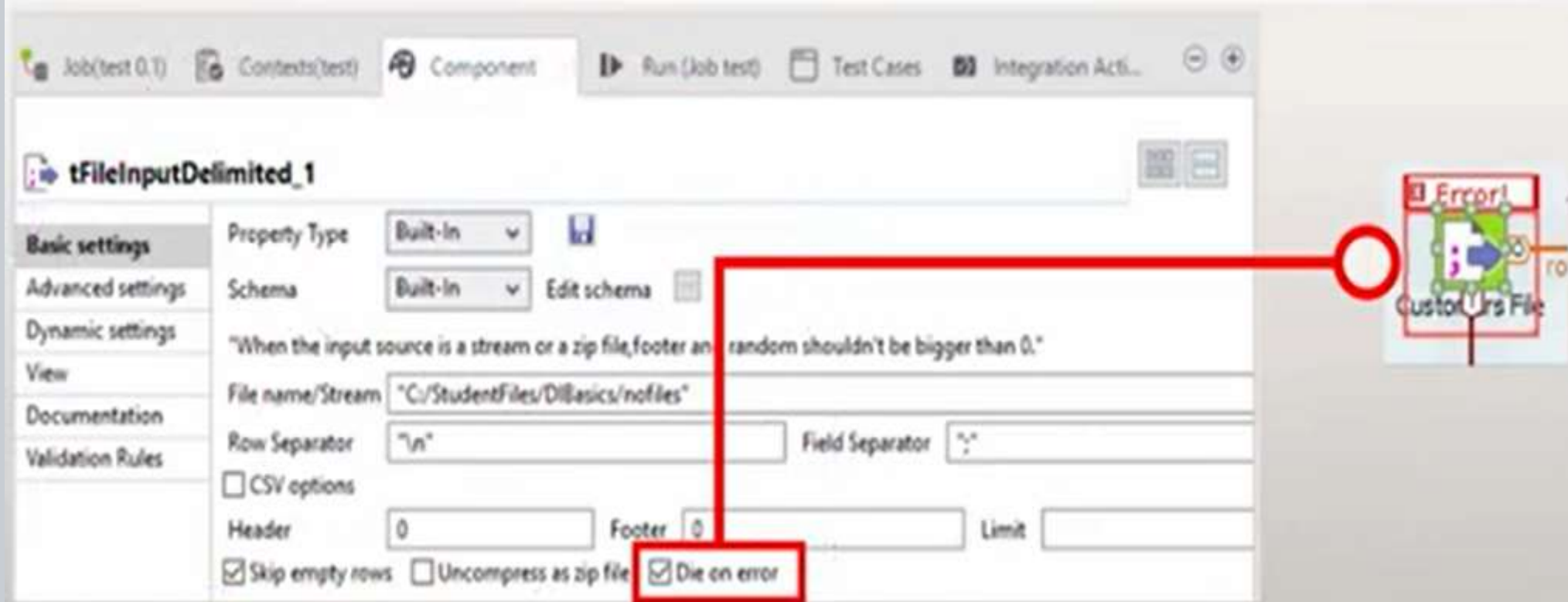
- Le Job s'arrête dès qu'il rencontre une erreur
- Le Job termine son exécution malgré l'erreur



Connexions aux DBs & Transformations

Gestion des erreurs

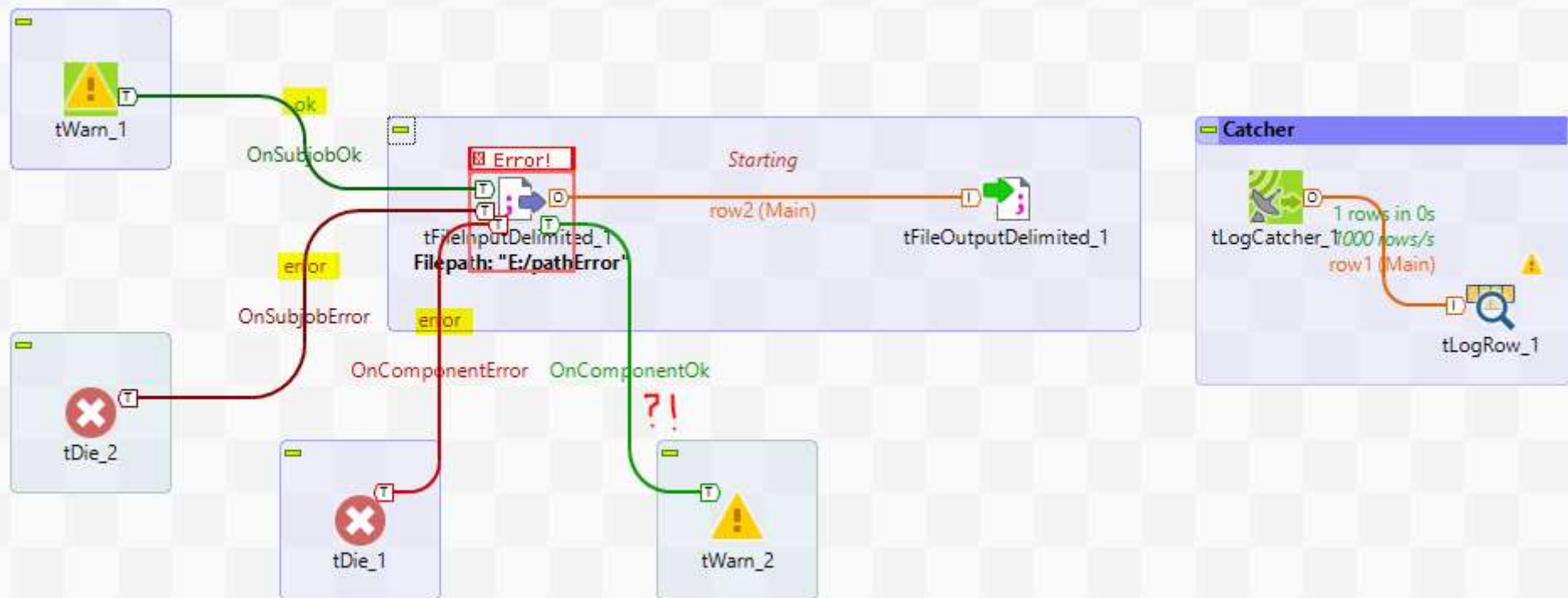
L'option **Die on error** est disponible pour de nombreux composants
Dans ce cas, l'exécution du job s'arrête si le composant génère une erreur



Connexions aux DBs & Transformations

Gestion des erreurs : tDie , tWarn , tLogCatcher

Les liens **Trigger** ne transportent pas de données, mais ils transfèrent le contrôle d'un composant à un autre



Connexions aux DBs & Transformations

Gestion des erreurs : tDie , tWarn , tlogCatcher

Avec tlogCatcher

Exécution

▶ Exécuter ■ Arrêter 🗑 Effacer

```
Starting job tLogCatcher1_Copy at 18:12 03/11/2025.
[statistics] connecting to socket on port 3890
[statistics] connected
2025-11-03 18:12:29|8Sji6X|8Sji6X|8Sji6X|REF_ATM_BNP|tLogCatcher1_Copy|demo|4|tWarn|tWarn_1|*****MSG from tWarn_1*****|42
Exception in component tFileInputDelimited_1 (tLogCatcher1_Copy)
java.io.FileNotFoundException: E:\pathError (Le chemin d'accès spécifié est introuvable)
  at java.base/java.io.FileInputStream.open0(Native Method)
  at java.base/java.io.FileInputStream.open(FileInputStream.java:219)
  at java.base/java.io.FileInputStream.<init>(FileInputStream.java:157)
  at java.base/java.io.FileInputStream.<init>(FileInputStream.java:112)
  at org.talend.fileprocess.TOSDelimitedReader.<init>(TOSDelimitedReader.java:88)
  at org.talend.fileprocess.FileInputDelimited.<init>(FileInputDelimited.java:164)
  at ref_atm_bnp.tlogcatcher1_copy_0_1.tLogCatcher1_Copy.tFileInputDelimited_1Process(tLogCatcher1_Copy.java:1502)
  at ref_atm_bnp.tlogcatcher1_copy_0_1.tLogCatcher1_Copy.tWarn_1Process(tLogCatcher1_Copy.java:1213)
  at ref_atm_bnp.tlogcatcher1_copy_0_1.tLogCatcher1_Copy.runJobInTOS(tLogCatcher1_Copy.java:2361)
  at ref_atm_bnp.tlogcatcher1_copy_0_1.tLogCatcher1_Copy.main(tLogCatcher1_Copy.java:2193)
*****MSG from tDie_1*****
2025-11-03 18:12:29|8Sji6X|8Sji6X|8Sji6X|REF_ATM_BNP|tLogCatcher1_Copy|demo|5|tDie|tDie_1|*****MSG from tDie_1*****|11
*****MSG from tDie_2*****
2025-11-03 18:12:29|8Sji6X|8Sji6X|8Sji6X|REF_ATM_BNP|tLogCatcher1_Copy|demo|5|tDie|tDie_2|*****MSG from tDie_2*****|22
2025-11-03 18:12:29|8Sji6X|8Sji6X|8Sji6X|REF_ATM_BNP|tLogCatcher1_Copy|demo|6|Java Exception|tFileInputDelimited_1|java.io.FileNotFoundException|
66 milliseconds
[statistics] disconnected
Job tLogCatcher1_Copy ended at 18:12 03/11/2025. [exit code = 22]
```

Sans tlogCatcher

Exécution

▶ Exécuter ■ Arrêter 🗑 Effacer

```
Starting job tLogCatcher1_Copy at 18:14 03/11/2025.
[statistics] connecting to socket on port 3930
[statistics] connected
Exception in component tFileInputDelimited_1 (tLogCatcher1_Copy)
java.io.FileNotFoundException: E:\pathError (Le chemin d'accès spécifié est introuvable)
  at java.base/java.io.FileInputStream.open0(Native Method)
  at java.base/java.io.FileInputStream.open(FileInputStream.java:219)
  at java.base/java.io.FileInputStream.<init>(FileInputStream.java:157)
  at java.base/java.io.FileInputStream.<init>(FileInputStream.java:112)
  at org.talend.fileprocess.TOSDelimitedReader.<init>(TOSDelimitedReader.java:88)
  at org.talend.fileprocess.FileInputDelimited.<init>(FileInputDelimited.java:164)
  at ref_atm_bnp.tlogcatcher1_copy_0_1.tLogCatcher1_Copy.tFileInputDelimited_1Process(tLogCatcher1_Copy.java:807)
  at ref_atm_bnp.tlogcatcher1_copy_0_1.tLogCatcher1_Copy.tWarn_1Process(tLogCatcher1_Copy.java:518)
  at ref_atm_bnp.tlogcatcher1_copy_0_1.tLogCatcher1_Copy.runJobInTOS(tLogCatcher1_Copy.java:1658)
  at ref_atm_bnp.tlogcatcher1_copy_0_1.tLogCatcher1_Copy.main(tLogCatcher1_Copy.java:1490)
*****MSG from tDie_1*****
*****MSG from tDie_2*****
29 milliseconds
[statistics] disconnected
Job tLogCatcher1_Copy ended at 18:14 03/11/2025. [exit code = 22]
```

!Warn!

Connexions aux DBs & Transformations

Logs, stats, flowMeters

◆ tStatCatcher

→ Capte les statistiques (durée, lignes traitées, etc.)

◆ tLogCatcher

→ Capte les erreurs et exceptions

◆ tFlowMeter / tFlowMeterCatcher

→ Capte le débit des lignes

Job(tLogCatcher1_Copy 0.1) Contexts(tLogCatcher1_Copy) Composant Exécuter(Job tLogCatcher1_Copy)

tLogCatcher1_Copy 0.1

Main: [Recharger depuis les paramètres du projet] [Enregistrer dans les paramètres du projet] ☐ Utiliser les paramètres du projet

Extra: ☒ Utiliser les statistiques (tStatCatcher) ☒ Utiliser les logs (tLogCatcher) ☒ Utiliser les volumes (tFlowMeterCatcher)

Stats & Logs: ☒ Dans la console

Version: ☒ Dans des fichiers

Chemin d'accès au fichier: "C:/dev/FormationTalend/JDD"

Nom du fichier de statistiques: "statsjob.log"

Nom du fichier de log: "job.log"

Nom du fichier de mesure: "flowmetterjob.log"

Encodage: UTF-8

☐ Dans la base de données

☒ Capturer les erreurs de l'exécutable ☒ Capturer les erreurs de l'utilisateur ☒ Capturer les alertes à l'utilisateur

Output (job.log):

```
1 2025-11-03 18:42:25;c96L31;c96L31;c96L31;REF_ATM_BNP;tLogCatcher1_Copy;demo;4;tWarn;tWarn_1;*****MSG from tWarn_1*****;42
2 2025-11-03 18:42:25;c96L31;c96L31;c96L31;REF_ATM_BNP;tLogCatcher1_Copy;demo;5;tDie;tDie_1;*****MSG from tDie_1*****;11
3 2025-11-03 18:42:25;c96L31;c96L31;c96L31;REF_ATM_BNP;tLogCatcher1_Copy;demo;5;tDie;tDie_2;*****MSG from tDie_2*****;22
4 2025-11-03 18:42:25;c96L31;c96L31;c96L31;REF_ATM_BNP;tLogCatcher1_Copy;demo;6;Java Exception;tFileInputDelimited_1;java.io.FileNotFoundException:E:\pathError (Le
5
```

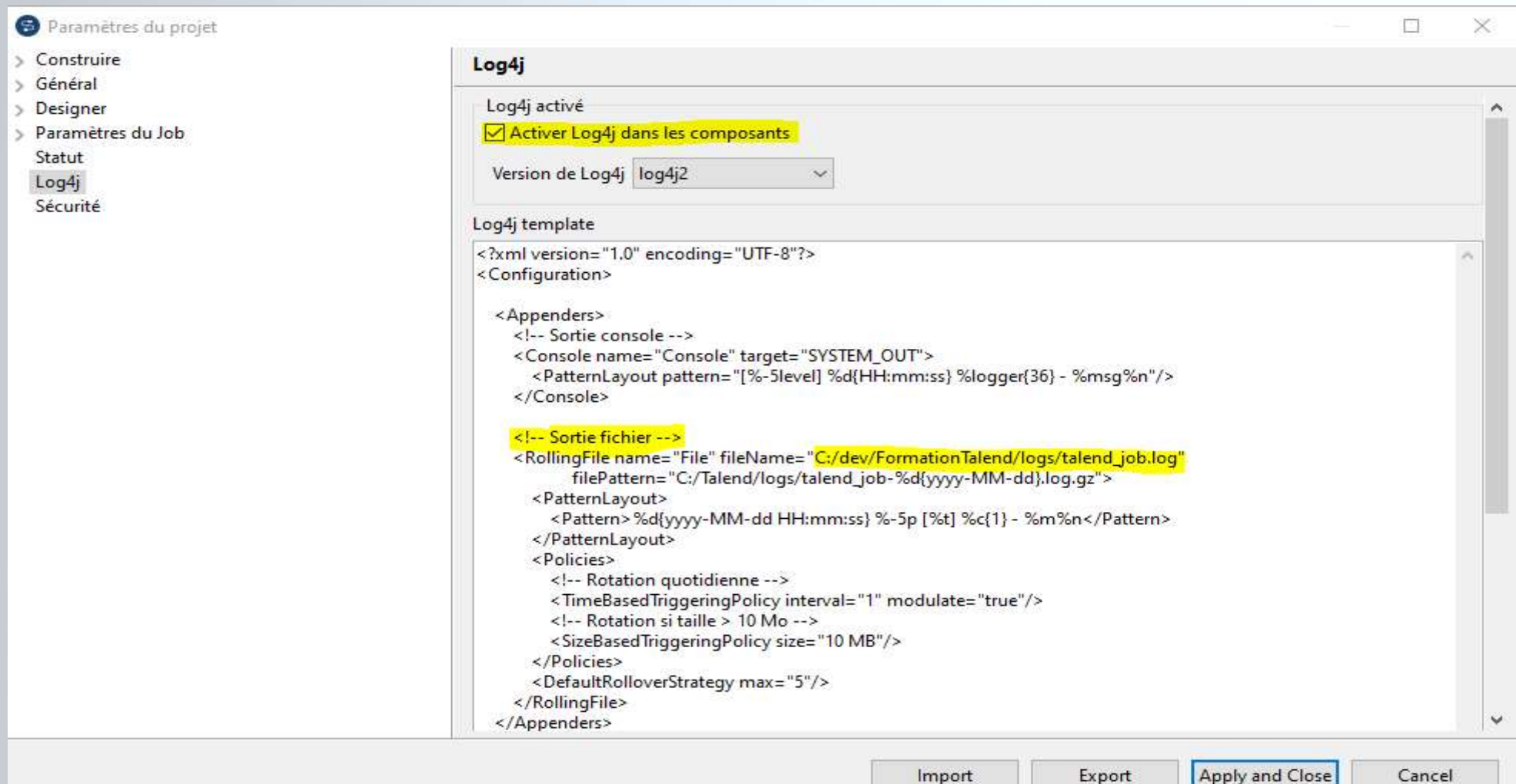
Output (statsjob.log):

```
1 2025-11-03 18:42:25;c96L31;c96L31;c96L31;17992;REF_ATM_BNP;tLogCatcher1_Copy; I-1LILjSEfCbZ-L321B8Ig;0.1;demo;;begin;;
2 2025-11-03 18:42:25;c96L31;c96L31;c96L31;17992;REF_ATM_BNP;tLogCatcher1_Copy; I-1LILjSEfCbZ-L321B8Ig;0.1;demo;;end;failure;127
3
```

Connexions aux DBs & Transformations

Activation Log4j dans Talend

- Dans les paramètres du projet , on pourra activer le log4j et mettre à jour la configuration , pour une orientation des logs vers un fichiers



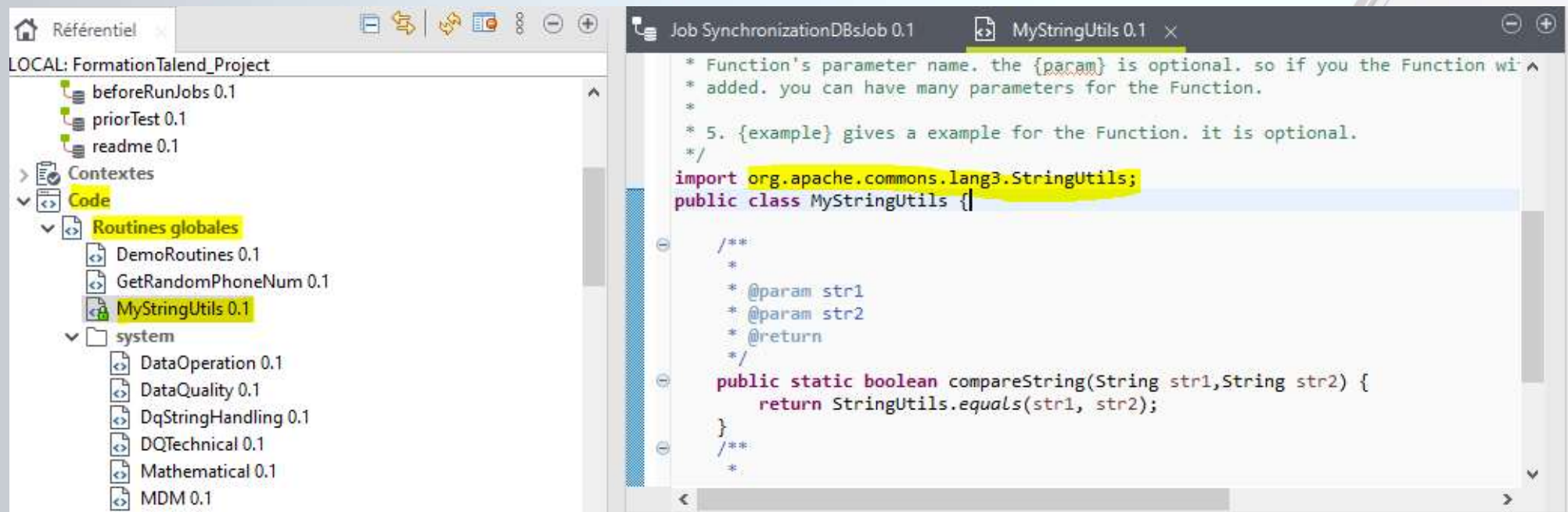
Atelier 2 : Synchronisation de données entre deux bases PostgreSQL

- En utilisant les composants tDBinput , tDBoutput, tMap, tFileOutputDelimited , tLogRow tDie, tWarn, tLogCatcher... , construit un job qui synchronise l'état d'une base de donnée de Backup avec celle primaire (exemple Table produit)
- 1) Logger les lignes traités
- 2) Implémenter une jointure entre les données Backup et Primaire
- 3) Utiliser une ou plusieurs tMap pour détecter/Filtrer les produits (Ajouté, Modifié, supprimé),
- 4) Mettre à niveau la DB Backup avec les « deltas » de la DB primaire,
- 5) Implémenter une gestion d'erreur transverse pour le job
- 6) Exporter les produits Supprimé dans un fichier CSV, pour un envoi au système de catalogue produits,
- 7) Fermer la connexion à la fin du job
- 8) Paramétrage dynamique via contextes des infos de connexion à la base et du fichier d'export des produits supprimés.

Réutilisation, Contextes et SubJobs

Customisation du code : Routine

- Une routine dans Talend est un code Java réutilisable (une classe utilitaire statique) que tu définis dans la partie "Code → Routines" du Repository
 - NB : possibilité d'importer des modules externes!
- ➔ Elle sert à centraliser du code Java que tu veux réutiliser dans plusieurs jobs



Réutilisation, Contextes et SubJobs

Customisation du code : Routines prédéfinis

- Importés et accessible automatiquement lors de la construction d'un nouveau job
- Les fonctions Talend

The screenshot displays the Talend Studio interface. On the left, the 'Routines' palette is open, showing a tree structure under 'LOCAL: REF_ATM_BNP'. The 'Routines' folder is expanded, revealing sub-folders like 'DemoRoutines 0.1' and 'system'. Under 'system', various routines are listed, with 'TalendDataGenerator 0.1' highlighted. The main editor on the right shows a Java code file named 'Job tSetGlobalVar 0.1'. The code includes a package declaration and a series of 'import' statements for various Talend routines and standard Java classes. The 'import routines.TalendDataGenerator;' line is highlighted in yellow.

```
// See the License for the specific language governing permissions and
// limitations under the License.

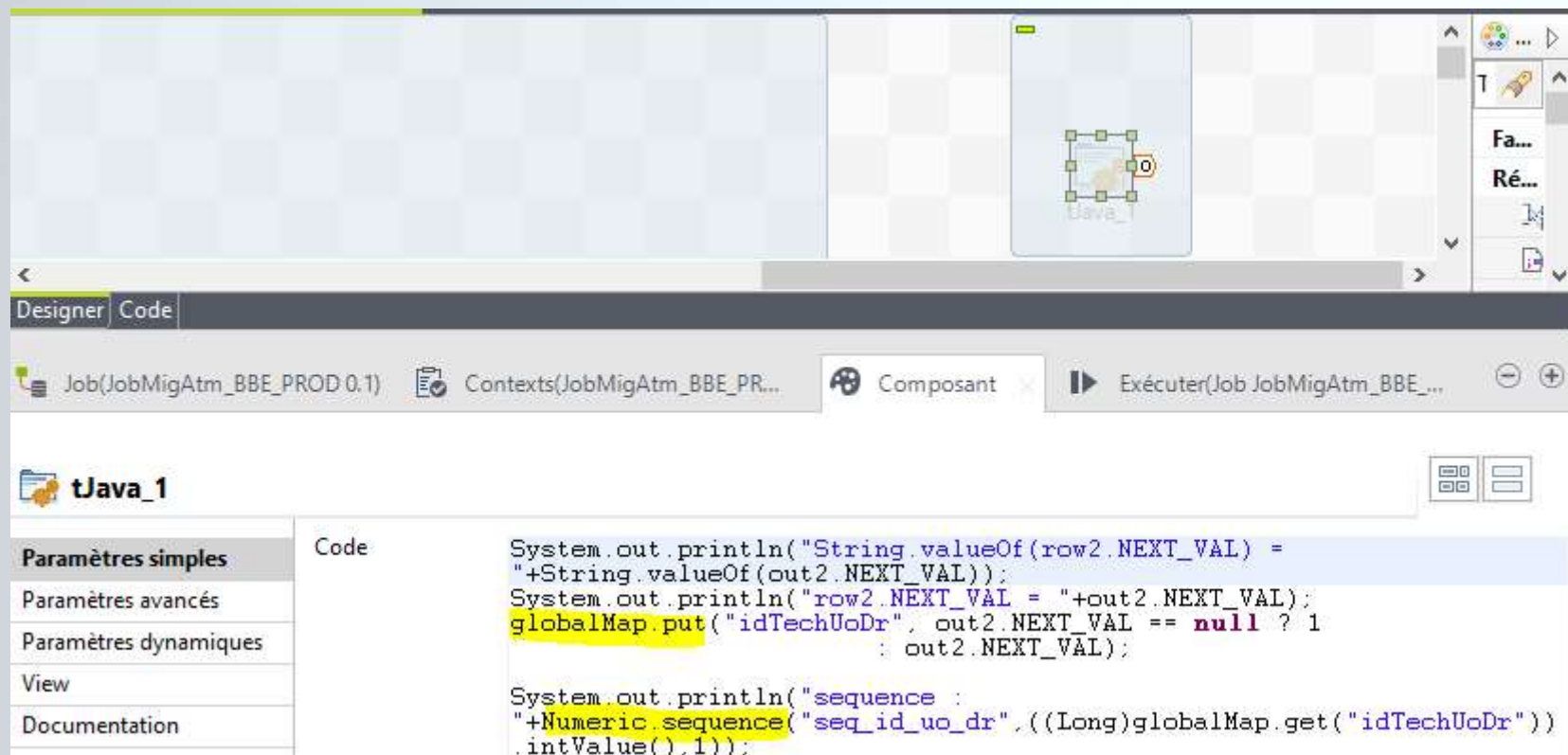
package ref_atm_bnp.tsetglobalvar_0_1;

import routines.Numeric;
import routines.DataOperation;
import routines.TalendDataGenerator;
import routines.TalendStringUtil;
import routines.TalendString;
import routines.StringHandling;
import routines.Relational;
import routines.TalendDate;
import routines.DemoRoutines;
import routines.GetRandomPhoneNum;
import routines.Mathematical;
import routines.system.*;
import routines.system.api.*;
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.List;
import java.math.BigDecimal;
import java.io.ByteArrayOutputStream;
import java.io.ByteArrayInputStream;
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.ObjectOutputStream;
import java.io.ObjectInputStream;
```


Réutilisation, Contextes et SubJobs

Customisation du code : tJava

- Le composant tJava, quand à lui :
 - ✓ Exécute du code Java directement dans un job (au runtime).
 - ✓ Est utile pour écrire du code **ponctuel, pas réutilisable** (une logique spécifique au job)



Réutilisation, Contextes et SubJobs

Customisation du code : tSetGlobalVar

➤ Le composant tSetGlobalVar :

- ✓ Sert à créer ou modifier des variables globales accessibles partout dans ton Job.
- ✓ Le tSetGlobalVar te permet de faire ça sans écrire de code Java

`globalMap.put("nom_variable", valeur);`

The screenshot shows the configuration of the **tSetGlobalVar_1** component. The **OnSubJobOk** event is connected to the **tMsgBox_1** component. The **Variables** table is visible, showing three global variables: **Name** (value: "YOUNG"), **FirstName** (value: "Angus"), and **BirthDate** (value: "31/03/1955").

Clé	Valeur
"Name"	"YOUNG"
"FirstName"	"Angus"
"BirthDate"	"31/03/1955"

Réutilisation, Contextes et SubJobs

Contextes : Création de contextes multiples

Possibilité 1 : Définir manuellement un contexte et l'alimenter avec les configurations (clé/valeur) nécessaires par environnement

Créer/Modifier un groupe de contextes
Étape 2 sur 2
Définir les contextes, variables et valeurs

	Name	Type	Comment	DEV		PROD		QLF	
				Value		Value		Value	
1	NbrInstances	int Integer		1	☐	5	☐	2	

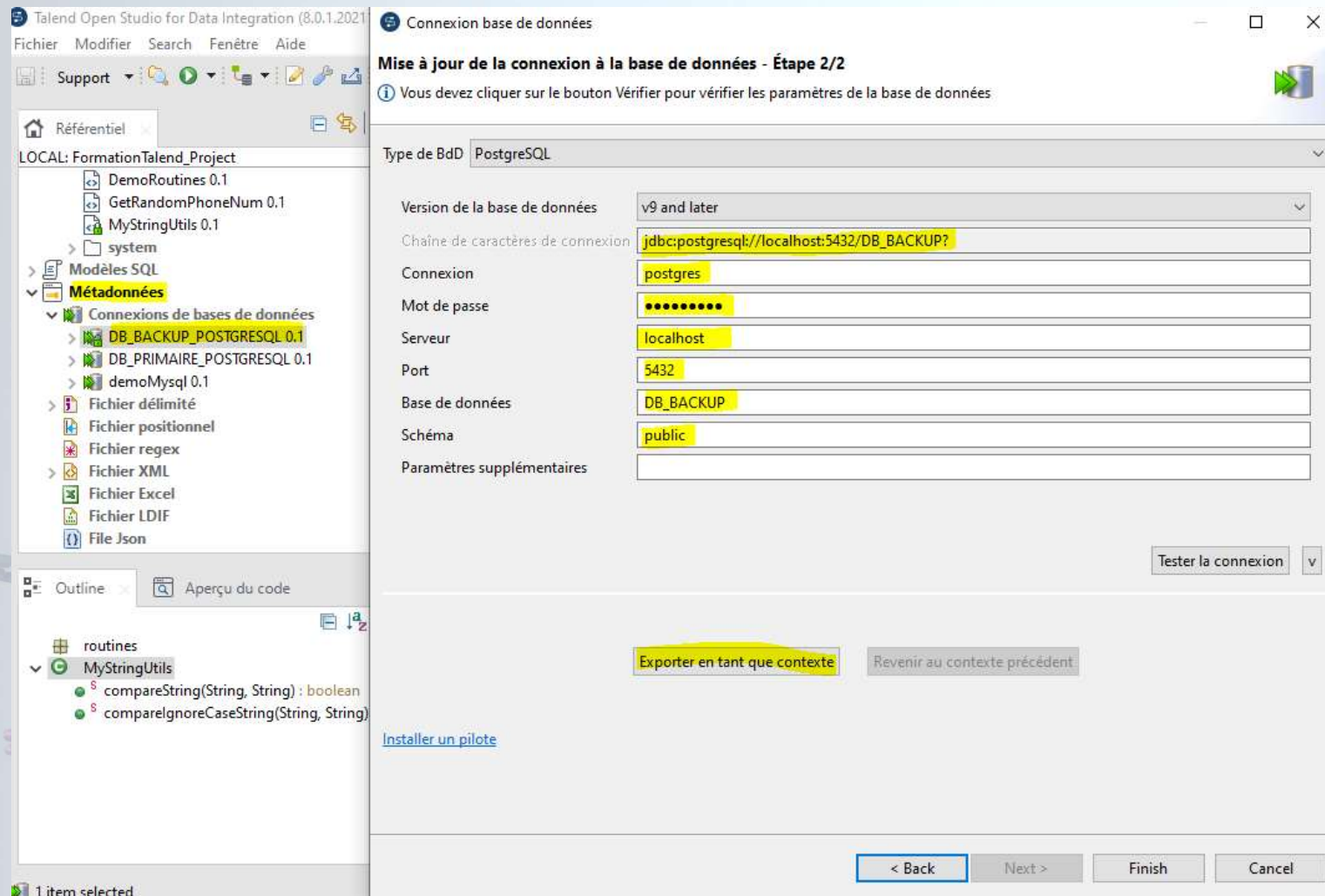
Environnement du contexte par défaut: DEV

Buttons: < Back, Next >, Finish, Cancel

Réutilisation, Contextes et SubJobs

Contextes : Création de contextes multiples

Possibilité 2 : importer directement un ensemble de paramètre et d'un seul coup depuis la configuration d'un métadonnées ou depuis la configuration d'un composants vers un contexte dédié !



Réutilisation, Contextes et SubJobs

Contextes : Création de contextes multiples

Créer/Réutiliser un groupe de contextes

Créer un contexte ou réutiliser le contexte existant

☒ Créer un contexte dans le référentiel

☐ Réutiliser un contexte existant dans le référentiel

< Back Next > Finish Cancel

Exporter en tant que contexte Revenir au contexte précédent

Créer/Réutiliser un groupe de contextes

Étape 1 sur 2

Il est déconseillé de laisser vide le champ Objectif.

Nom DB_BACKUP_POSTGRESQL_CONTEXTE

Objectif

Description

Créé par : user@talend.com

Verrouillé par

Version 0.1 M m

Statut

Chemin d'accès Sélectionner

< Back Next > Finish Cancel

Créer/Réutiliser un groupe de contextes

Étape 2 sur 2

Définir les contextes, variables et valeurs

	Name	Type	Comment	DEV	
				Value	
1					
2	DB_BACKUP_POSTGRESQL_Port	String		5432	☐
3	DB_BACKUP_POSTGRESQL_Database	String		DB_BACKUP	☐
4	DB_BACKUP_POSTGRESQL_AdditionalParams	String			☐
5	DB_BACKUP_POSTGRESQL_Server	String		localhost	☐
6	DB_BACKUP_POSTGRESQL_Login	String		postgres	☐
7	DB_BACKUP_POSTGRESQL_Password	Password		*****	☐
8	DB_BACKUP_POSTGRESQL_Schema	String		public	☐

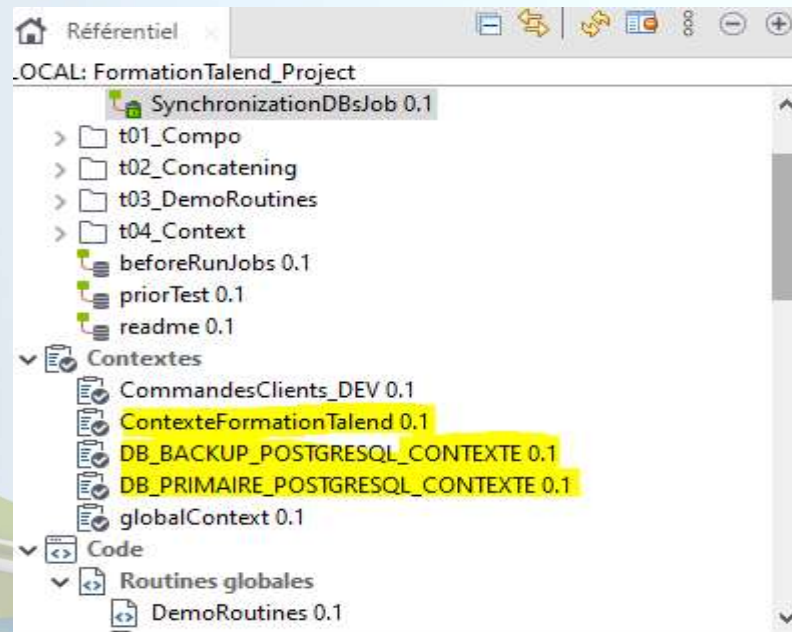
Environnement du contexte par défaut DEV

< Back Next > Finish Cancel

Réutilisation, Contextes et SubJobs

Contextes : Création de contextes multiples

- Les noms donnés aux contextes servent comme préfixe pour les clés de chaque source de donnée , ce sont des entités logique pendant le développement.
- Lors de la construction du **Job final** , tous les clés/valeurs des différents contextes sont regroupés dans un seul fichier properties par environnement!



Ce PC > Disque local (C:) > dev > SynchronizationDBsJob_0.1 > SynchronizationDBsJob > formationtalend_project > synchronizationdbsjob_0_1 > contexts

Nom	Modifié le	Type	Taille
Default.properties	04/11/2025 01:37	Fichier source Pro...	1 Ko
DEV.properties	04/11/2025 01:37	Fichier source Pro...	1 Ko
QLF.properties	04/11/2025 01:37	Fichier source Pro...	1 Ko

Structuration des projets & Orchestration

tPrejob, tPostJob



➤ tPreJob — Avant le Job principal

➔ Rôle : Exécuter des actions avant le début du flux principal du job :

- ✓ Initialiser des contextes ou des variables globales
- ✓ Ouvrir une connexion à une base de données
- ✓ Charger un fichier de configuration
- ✓ Créer des dossiers ou fichiers temporaires
- ✓ Vérifier l'existence d'un fichier ou d'une table avant traitement

➤ tPostJob — Après le Job principal

➔ Rôle : Exécuter des actions après la fin du flux principal (qu'il y ait erreur ou pas) :

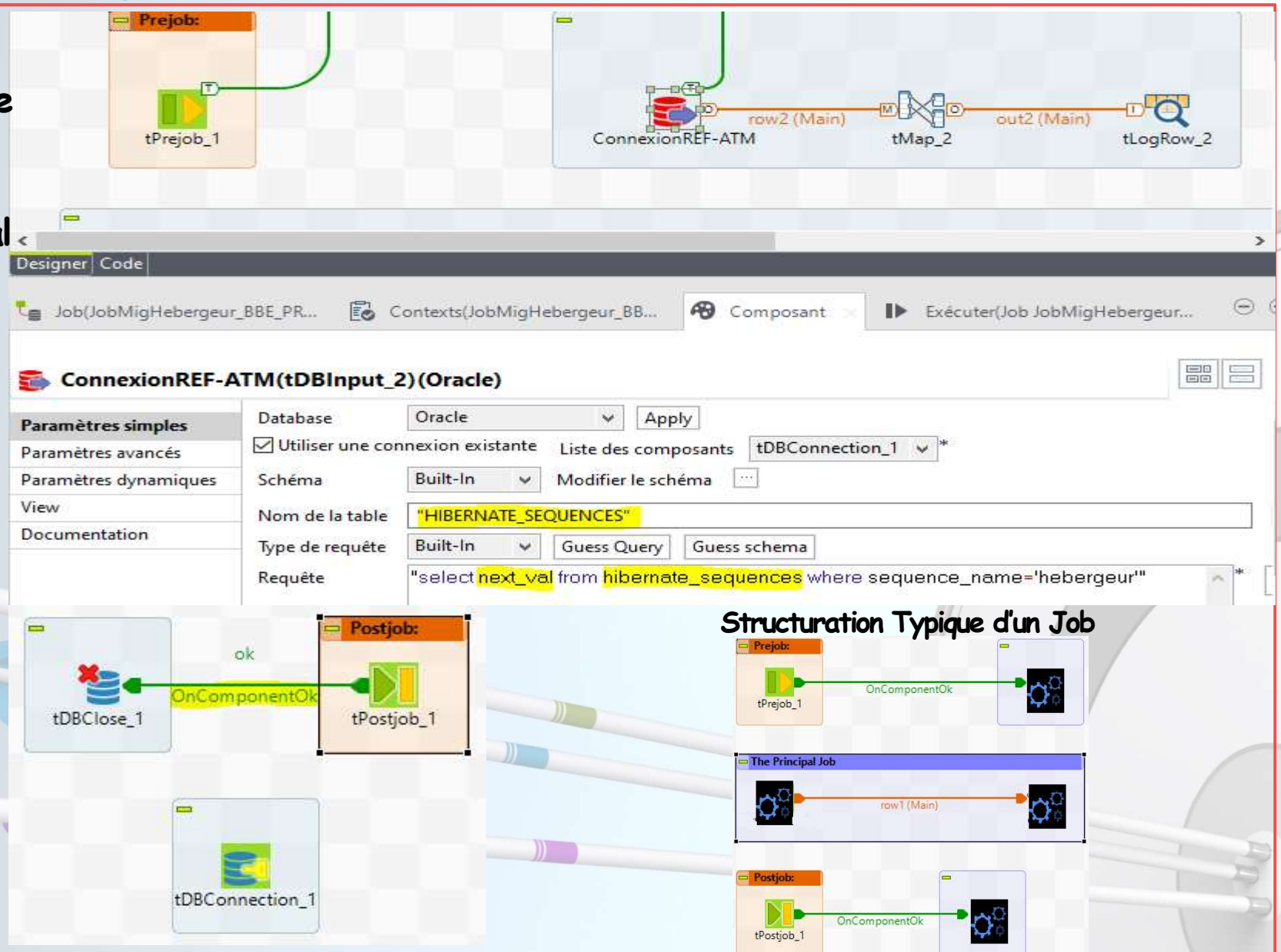
- ✓ Fermer les connexions à la base de données (tDBCclose)
- ✓ Envoyer un mail de notification
- ✓ Supprimer ou archiver des fichiers temporaires
- ✓ Écrire un résumé d'exécution ou un fichier de log
- ✓ Libérer des ressources (sessions FTP, etc.)

Structuration des projets & Orchestration

tPrejob, tPostJob

Préparation du numéro de séquence suivant pour une insertion ultérieure dans le job principal (cas de gestion des PKs par Hibernate)

Fermeture de connexion à la fin du job ou commit de transaction



Structuration des projets & Orchestration

Orchestration : tRunJob & onSubJobOk

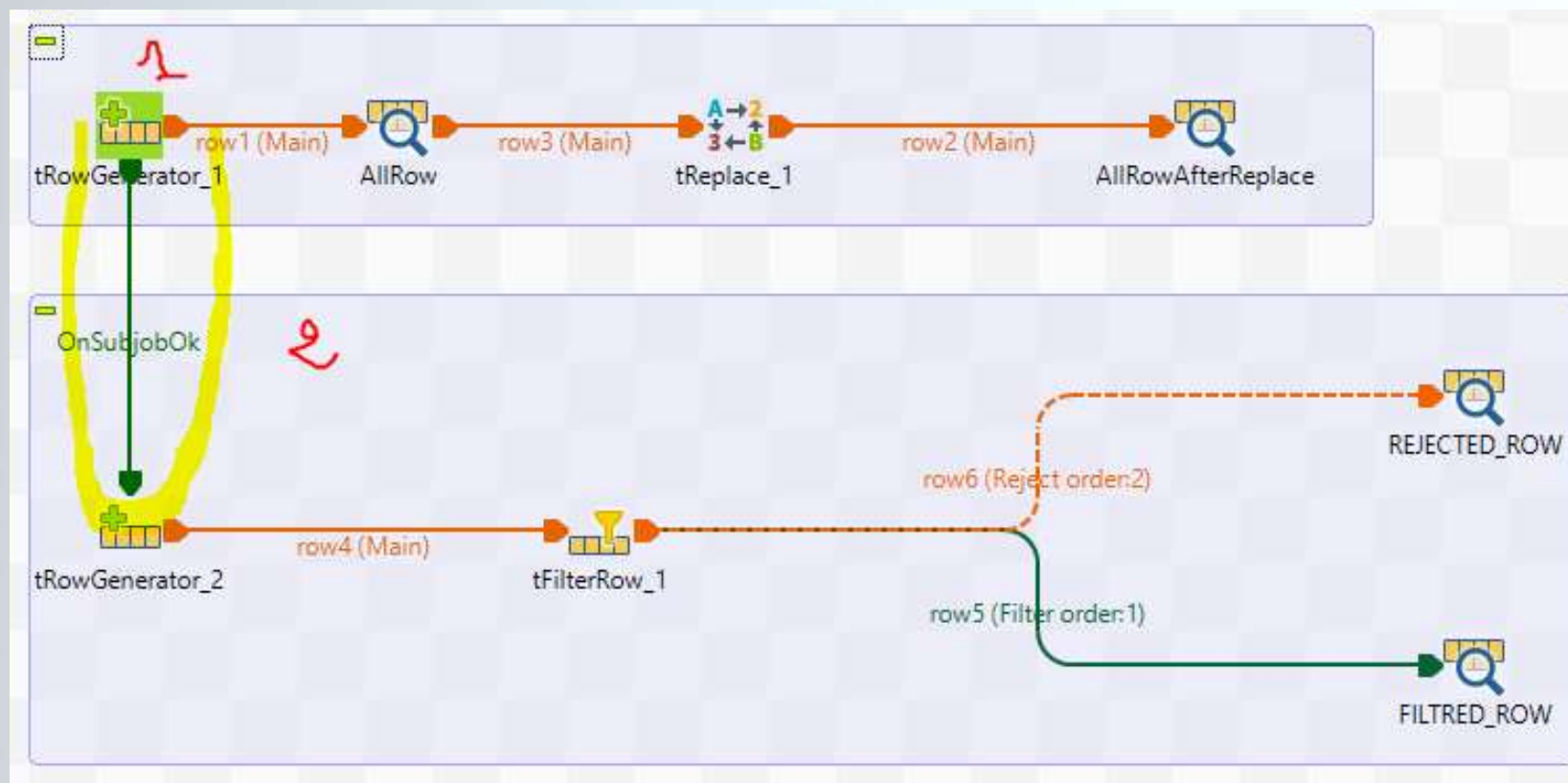
- **tRunJob** permet de lancer un autre job Talend depuis un job principal.
- Le job lancé devient ce qu'on appelle un sous-job.
- **Structuration** : Cela permet de réutiliser des jobs existants sans copier-coller leur contenu.
- C'est un composant principale pour **l'orchestration** des Jobs

The screenshot displays the Talend Studio interface for job orchestration. On the left, the project tree shows a hierarchy where 'tRunJob' is expanded, revealing 'jobChildren' and 'jobParent'. The central workspace shows a job design with a 'tMsgBox_1' component (labeled 'JobParent is done! Press OK to launch the jobChildren') connected to a 'tRunJob_1' component via a 'Then Run' relationship. A callout bubble highlights the 'job children' sub-job configuration, which includes a 'tMsgBox_1' component with the message 'context.MESSAGE'. The bottom pane shows the configuration for 'tRunJob_1', where the 'Job' field is set to 'jobChildren' and the 'Version' is 'Latest'. The 'Contexte' is set to 'Default'.

Structuration des projets & Orchestration

Orchestration : tRunJob & onSubJobOk

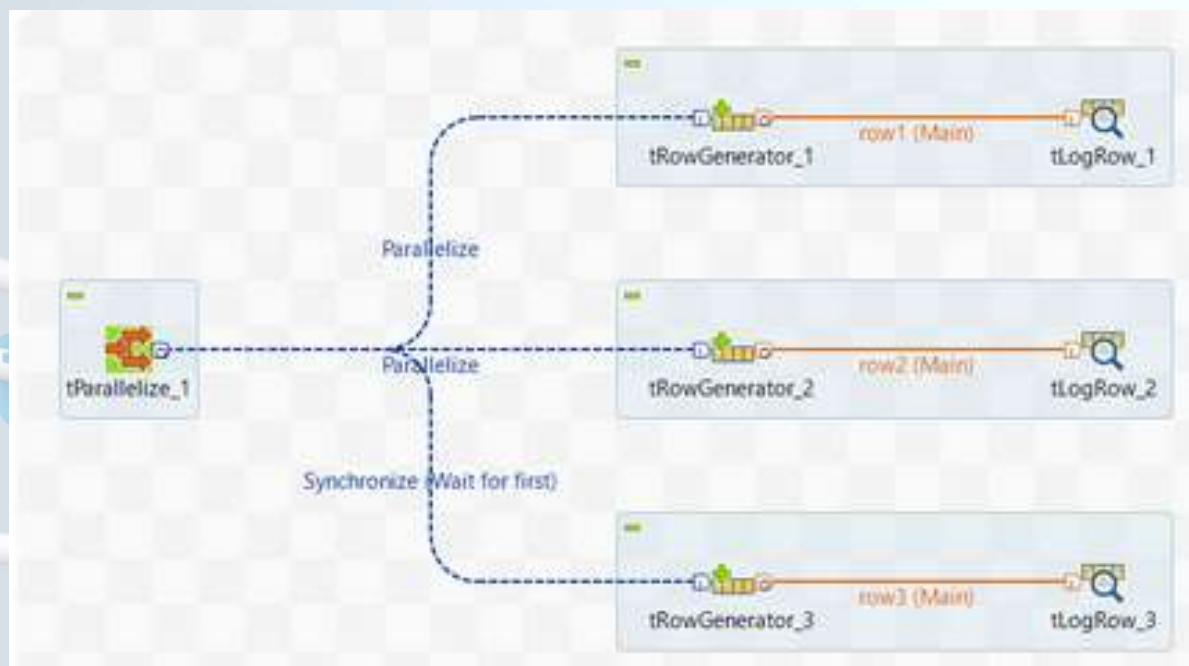
- Le trigger **onSubJobOk** permet aussi d'orchestrer des sous jobs dans le même fichier du job principal
- La bonne exécution du premier sous Job conditionne l'exécution du deuxième sous Job



Structuration des projets & Orchestration

Orchestration : tParallelize

- Le composant **tParallelize** est utilisé pour exécuter plusieurs sous-jobs ou tâches en parallèle, plutôt qu'en séquence.
- Chaque sous-job est exécuté dans un thread séparé.
- tParallelize est utile seulement si les sous-jobs n'ont **pas de dépendance entre eux**.
- Le composant fait partie de la version payante de Talend.



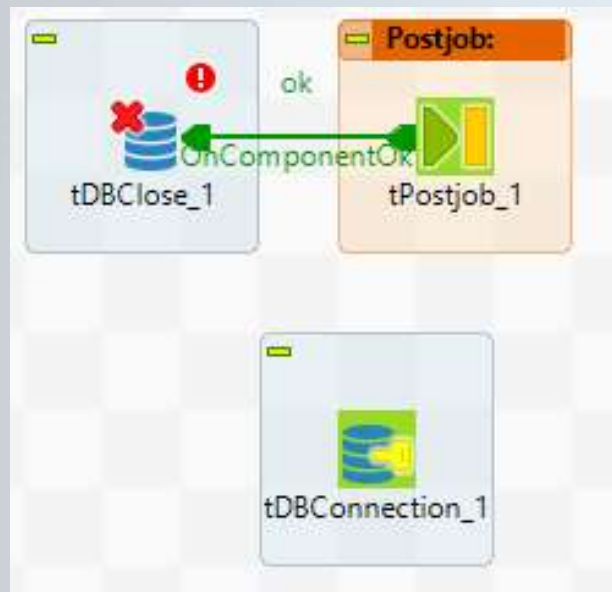
Atelier 3 : Structuration & Orchestration de job + création de context

- Point de départ : Job de synchronisation résultat de l'atelier 2
- En se basant/copiant ce Job et en utilisant en plus les composants/notions de tPrejob, tPostJob, routine, contextes multiples, repository metadata, effectuez les refactorings suivants :
 - 1) Ajouter un tPreJob Pour logger le début du Job et un tPostJob pour fermer les connexions DB.
 - 2) Orchestrer les deux sous-jobs par un lien logique (exécution conditionnelle)
 - 3) Exporter le paramétrage des sources de données dans des contextes dédiés, définissez 3 environnement (DEV,PROD, QLF)
 - 4) Centraliser les schémas des input/output de données dans des métadatas
 - 5) Implémenter une routine qui permet de faire les comparaisons de Strings avec l'API « Apache commons Lang », et l'utiliser dans le Job

Gestion des transactions, Chargement, Performance et Debug

tDBClose , tDBConnection

- Le composant tDBConnection sert à établir une connexion unique à une base de données au début d'un job Talend, afin que plusieurs autres composants (requêtes, insertions, mises à jour, etc.) puissent réutiliser cette même connexion.
- tDBClose , permet de fermer la connexion à la fin du Job , et du coup libérer les ressources DB.



A screenshot of the Talend Designer interface. The top part shows a job canvas with a component labeled 'produit' and a data flow. Below the canvas, there's a 'Designer' tab and a 'Code' tab. The 'Designer' tab is active, showing a job configuration for 'tDBClose_1 (PostgreSQL)'. The configuration includes a 'Database' dropdown set to 'PostgreSQL' and a 'Liste des composants' dropdown set to 'tDBConnection_1'. The 'tDBConnection_1' component is highlighted in the list. The interface also shows a 'Postjob' section with 'tPostjob_1' and a 'tDBClose_1' component. A yellow arrow points from the 'tDBClose_1' component in the job canvas to the 'tDBClose_1' component in the configuration panel.

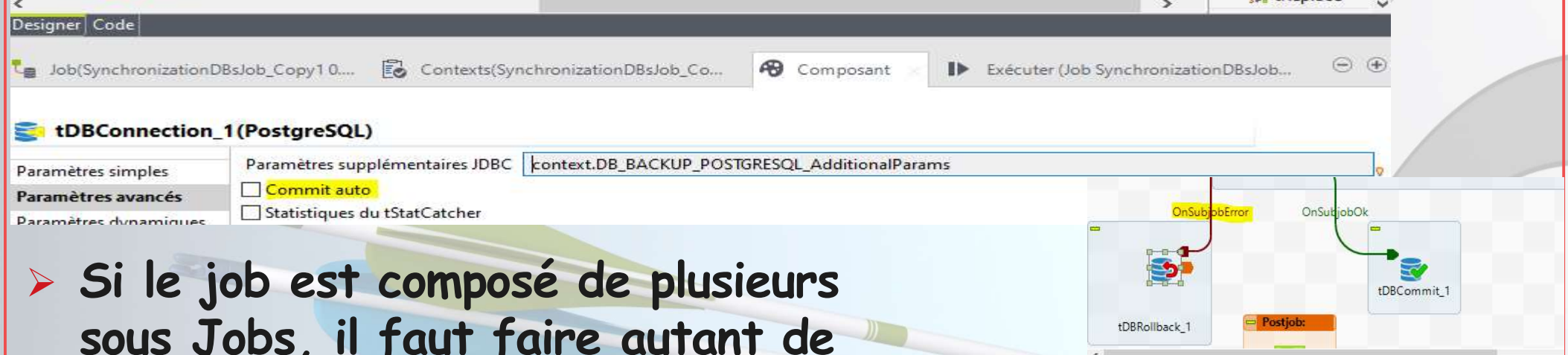
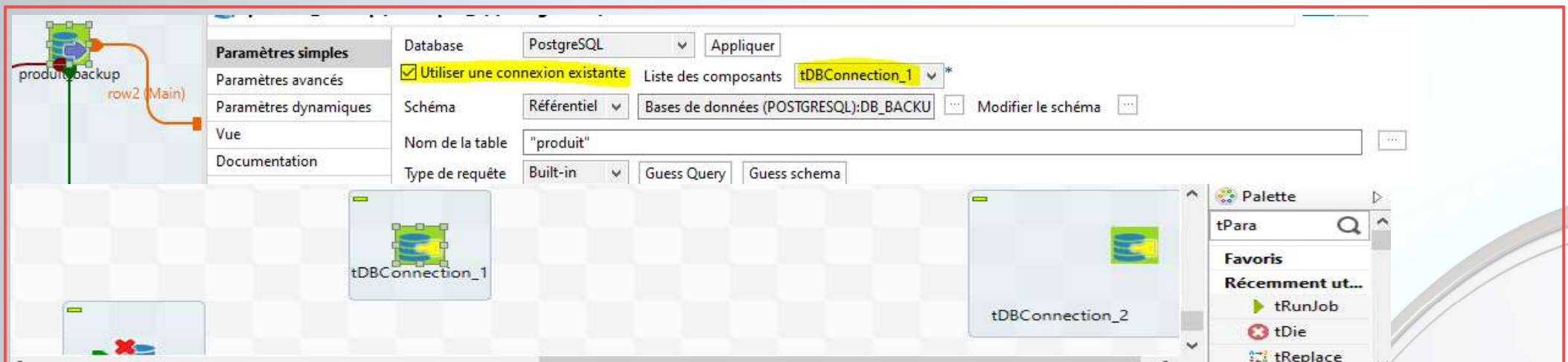
Gestion des transactions, Chargement, Performance et Debug

tDBCommit & tDBRollback

- Afin de gérer efficacement la transaction au niveau d'un Job donné il faut respecter les éléments suivants :
- 1. Une seul élément tDBConnection Créé (par SDD) et référencé par l'ensemble des composants tDBCommit ,tDBRollback (tDBOutput, tDBInput tDBUpdate, etc.).
- 2. Option "Auto Commit" doit être désactivée si tu veux gérer toi-même les transactions.
- 3. Dans l'onglet "Advanced settings" des composants DB, coche Use an existing connection→ choisis le tDBConnection créé.
- 4. tDBCommit / tDBRollback N'ont pas de flux "Main" : ils sont déclenchés via des triggers (OnSubjobOk, OnSubjobError).Ils utilisent la même connexion.
- 5. Cocher « Fermer la connexion » dans tDBCommit / tDBRollback

Gestion des transactions, Chargement, Performance et Debug

tDBCommit & tDBRollback



Si le job est composé de plusieurs sous Jobs, il faut faire autant de tDBRollback que de sous Jobs

Un seul tDBCommit à la fin

Gestion des transactions, **Chargement, Performance** et Debug tDBBulkExec

- Le composant tDBBulkExec sert à charger un grand volume de données dans une base de données de manière massive (bulk), en utilisant les mécanismes natifs de la base (COPY, LOAD DATA, SQL*Loader...).
- il permet un import performant et très rapide de données **depuis un fichier plat** (CSV, TXT, etc.) vers **une table** de la base.
- Le fichier doit correspondre exactement à la structure de la table (ordre, types, séparateurs).

Gestion des transactions, **Chargement, Performance** et Debug tDBBulkExec

The screenshot displays the configuration interface for the **tDBBulkExec_1 (PostgreSQL)** component within a job designer. The interface is divided into several sections:

- Designer View:** Shows a workflow with components **CommandesClients** and **tDBBulkExec_1** connected by an **OnSubjobOk** event. A **DB_BACKUP_POSTGRESQL** component is also visible.
- Job Information:** The job is named **Job(ChargementEnMasse 0.1)** and is associated with the context **Contexts(ChargementEnMasse)**. The component is **Composant** and the action is **Exécuter (Job ChargementEnMasse)**.
- Configuration Parameters:**
 - Database:** PostgreSQL
 - Utiliser une connexion existante:** Checked
 - Liste des composants:** tDBConnection_1 - DB_BACKUP_POSTGRESQL
 - Table:** "produit"
 - Action sur la table:** Par défaut
 - Nom de fichier:** context.CommandesClients_File
 - Schéma:** Built-in
- Advanced Parameters (Paramètres avancés):**
 - Action sur les données:** Insertion de masse
 - Copie l'OID pour chaque ligne:** Unchecked
 - Contient une ligne d'en-tête avec le nom de chaque colonne du fichier:** Unchecked
 - Utiliser le fichier local pour la copie (pour un serveur de base de données 8.2 ou plus récente):** Unchecked
 - Type de fichier:** Fichier CSV
 - Chaine de caractères nulle:** ""
 - Champs terminés par:** ","
 - Caractère d'échappement:** \
 - Entourage du texte:** ""
 - Utiliser l'option standard_conforming_string:** Checked
 - Forcer les colonnes à n'être pas nulles:** Unchecked
- Dynamic Parameters (Paramètres dynamiques):**
 - Column:** (Empty field)
 - Forcer à n'être pas null:** Unchecked

Gestion des transactions, **Chargement, Performance** et Debug tDBBulkExec

Le gain de temps entre le chargement en bulk et le chargement unitaire est presque le **double**! Pour le même fichier (~100 lignes)

Job ChargementEnMasse

Exécution simple
Exécution en mode Debug
Paramètres avancés
Cible d'exécution
Exécution pour la mémoire

Exécution

Exécuter Arrêter Effacer

Démarrage du Job ChargementEnMasse à 02:14 05/11/2025.
[statistics] connecting to socket on port 3576.
[statistics] connected
258 milliseconds
[statistics] disconnected
Job ChargementEnMasse terminé à 02:14 05/11/2025. [Code ,

87 rows in 0,03s
3107,14 rows/s
row1 (Main)

87 rows in 0,04s
2071,43 rows/s
out1 (Main)

DB_BACKUP_POSTGRESQL DB_BACKUP_POSTGRESQL

Job ChargementUnitaire

Exécution simple
Exécution en mode Debug
Paramètres avancés
Cible d'exécution
Exécution pour la mémoire

Exécution

Exécuter Arrêter Effacer

Démarrage du Job ChargementUnitaire à 13:09
[statistics] connecting to socket on port 35
[statistics] connected
343 milliseconds
[statistics] disconnected
Job ChargementUnitaire terminé à 13:09 05/11

Default
Nom
DB_BACKUP_POST
DB_BACKUP_POST
DB_BACKUP_POST
DB_BACKUP_POST
DB_BACKUP_POST
DB_BACKUP_POST
DB_BACKUP_POST

Gestion des transactions, Chargement, Performance et **Debug**

Debugger un Job : breakpoints, trace mode

Le debugge SUR TOS se fait de la même manière que le debug sur Eclipse avec des points d'arrêts ,un lancement en mode Debug et une perspective de debug dédiée

The screenshot displays the Eclipse IDE interface. The top toolbar includes icons for file operations and debugging. The package explorer on the left shows the project structure. The editor window displays the source code of a Java job, with a breakpoint set on line 15787. The bottom left pane shows the 'Job CsvToCsvTransformationJob' view, where 'Exécution en mode Debug' is selected. A 'Confirm Perspective Switch' dialog is open, providing information about the Debug perspective and asking for confirmation to switch.

```
15786 public static void main(String[] args) {
15787     final CsvToCsvTransformationJob CsvToCsvTransformationJobClass = new CsvToCsvTransformationJob();
15788
15789     int exitCode = CsvToCsvTransformationJobClass.runJobInTOS(args);
15790     if (exitCode == 0) {
```

Job CsvToCsvTransformationJob

- Exécution simple
- Exécution en mode Debug**
- Paramètres avancés
- Cible d'exécution
- Exécution pour la mémoire

Debug

Débogage Java Arrêter

Démarrage du Job
CsvToCsvTransformationJob
à 19:33 05/11/2025.

Confirm Perspective Switch

This kind of launch is configured to open the Debug perspective when it suspends.

This Debug perspective supports application debugging by providing views for displaying the debug stack, variables and breakpoints.

Switch to this perspective?

☐ Remember my decision

Switch No

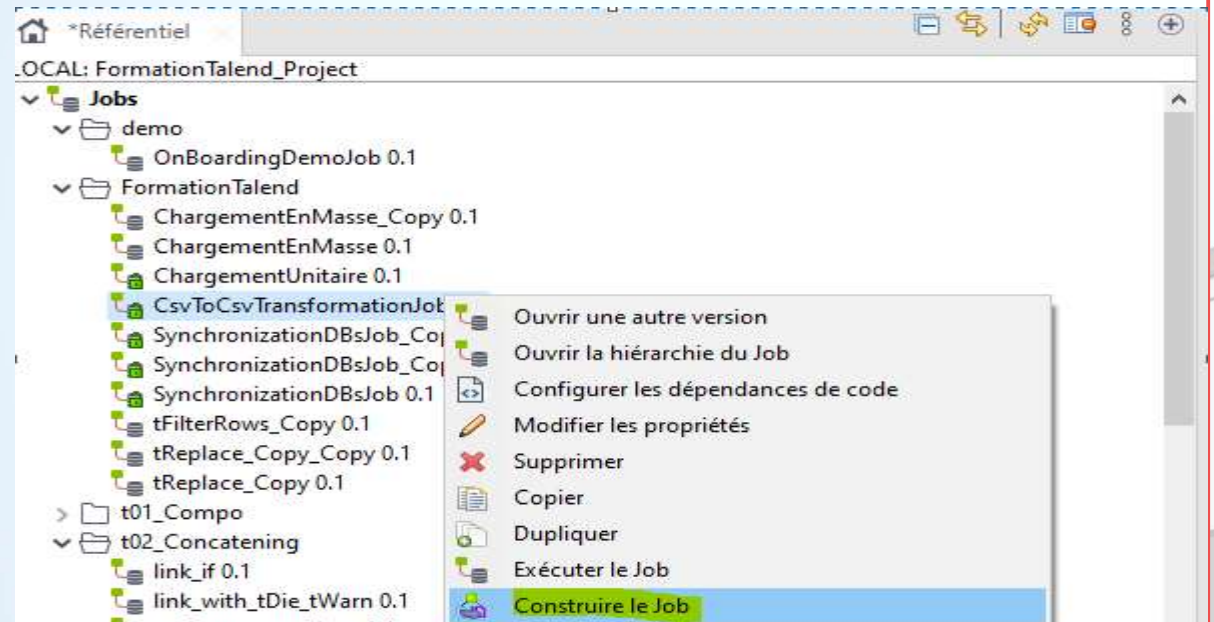
Atelier 4 : Gestion transaction & Chargement massif, dans job de synchronisation

- Point de départ : Job résultat de l'atelier 3
- En se basant/copiant ce Job et en utilisant en plus les composants `tDBConnection`, `tDBCommit`, `tDBRollback`, `tDBBulkExec`, `tMap`, `tFileInputDelimited`... effectuez les refactorings suivants :
 - 1) Ajouter deux `tDBConnection` transactionnels pour la primaire et la backup et associer tous les composants DB à ces composants.
 - 2) En cas d'erreur Rollbacker la transaction pour tous sous-job, via `tDBRollback`, et `tDBCommit` à la fin.
 - 3) Ajouter un contexte pour le paramétrage des chemins des fichiers de (logs , statistiques ,flowMetter), et configurer le job pour utiliser ces fichiers et collecter ces logs via (`tLogCatcher`, `tStatCatcher` , `tFlowMetterCatcher`)
 - 4) Créer un Job pour charger massivement la base Backup avec les nouveaux Produits (produits.csv 1000 prodits) , via `tDBBulkExec`
 - 5) Créer un Job pour charger la base primaire transactionnellement depuis le même fichier, avec `tFileInputDelimited` et tmap `tDBOutput`...
 - 6) Comparez les temps d'exécution

Industrialisation & Déploiement d'un projet Talend

Construction d'un Jar Exécutable

- Depuis le menu contextuel d'un Job
- Génération d'un package *.zip qui contient l'ensemble des exécutables, dépendances et contextes



Vers le fichier archive : C:\dev\CsvToCsvTransformationJob_0.1.zip Browse...

Job Version
Sélectionnez la version de Job 0.1

Type de construction
Sélectionner le type de construction Job standalone ☐ Extraire le fichier ZIP

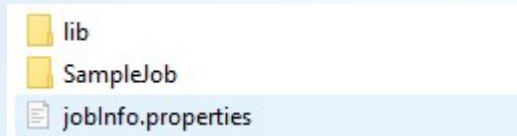
Options
☒ Interpréteur de commande Tous
☒ Scripts de contexte Unix Windows ☐ Appliquer le contexte aux Jobs enfants
☐ Écraser les valeurs des paramètres
☒ Éléments
☒ Sources Java

ation au format HTML
s

Industrialisation & Déploiement d'un projet Talend

Construction d'un Jar Exécutable

➤ Résultats SampleJob_0.1.zip :



C > Disque local (C:) > dev > SampleJob_0.1 > SampleJob

Nom	Modifié le	Type	Taille
formationtalend_project	07/11/2025 17:01	Dossier de fichiers	
log4j2.xml	07/11/2025 17:01	Microsoft Edge H...	2 Ko
samplejob_0_1.jar	07/11/2025 17:01	Fichier JAR	40 Ko
SampleJob_run.bat	07/11/2025 17:01	Fichier de comman...	1 Ko
SampleJob_run.ps1	07/11/2025 17:01	Script Windows P...	1 Ko
SampleJob_run.sh	07/11/2025 17:01	Shell Script	1 Ko

C > Disque local (C:) > dev > SampleJob_0.1 > SampleJob > formationtalend_project > samplejob_0_1 > contexts

Nom	Modifié le	Type	Taille
Default.properties	07/11/2025 16:53	Fichier source Pro...	1 Ko
DEV.properties	07/11/2025 16:53	Fichier source Pro...	1 Ko
QLF.properties	07/11/2025 16:53	Fichier source Pro...	1 Ko

Industrialisation & Déploiement d'un projet Talend

CI/CD Talend : Automatisation Construction Job

- Dans Talend Open Studio (TOS), il n'existe aucun moyen officiel de construire un job depuis la ligne de commande.
- Le moteur de construction des jobs Talend est intégré au Talend **CommandLine** (ou "CommandLine Interface"), qui est une application **payante** séparée du Studio.
- Cette application n'est distribuée que dans les versions **Enterprise / Data Fabric / Cloud**.
- TalendCommandLine est l'outil en ligne de commande qui permet d'automatiser les actions que tu ferais à la main dans le Studio :
 - ✓ se connecter à un projet,
 - ✓ construire un job,
 - ✓ déployer un job,
 - ✓ gérer les contextes, etc.

```
TalendCommandLine.sh \ buildJob -pj MonProjet -jn premier_job -jr 0.1 -pt  
standalone -dd /opt/talend/builds -af zip
```

➤ pipeline CI/CD Jenkins

- CI/CD Jenkins se base sur cette outil pour automatiser un pipeline complet

Industrialisation & Déploiement d'un projet Talend

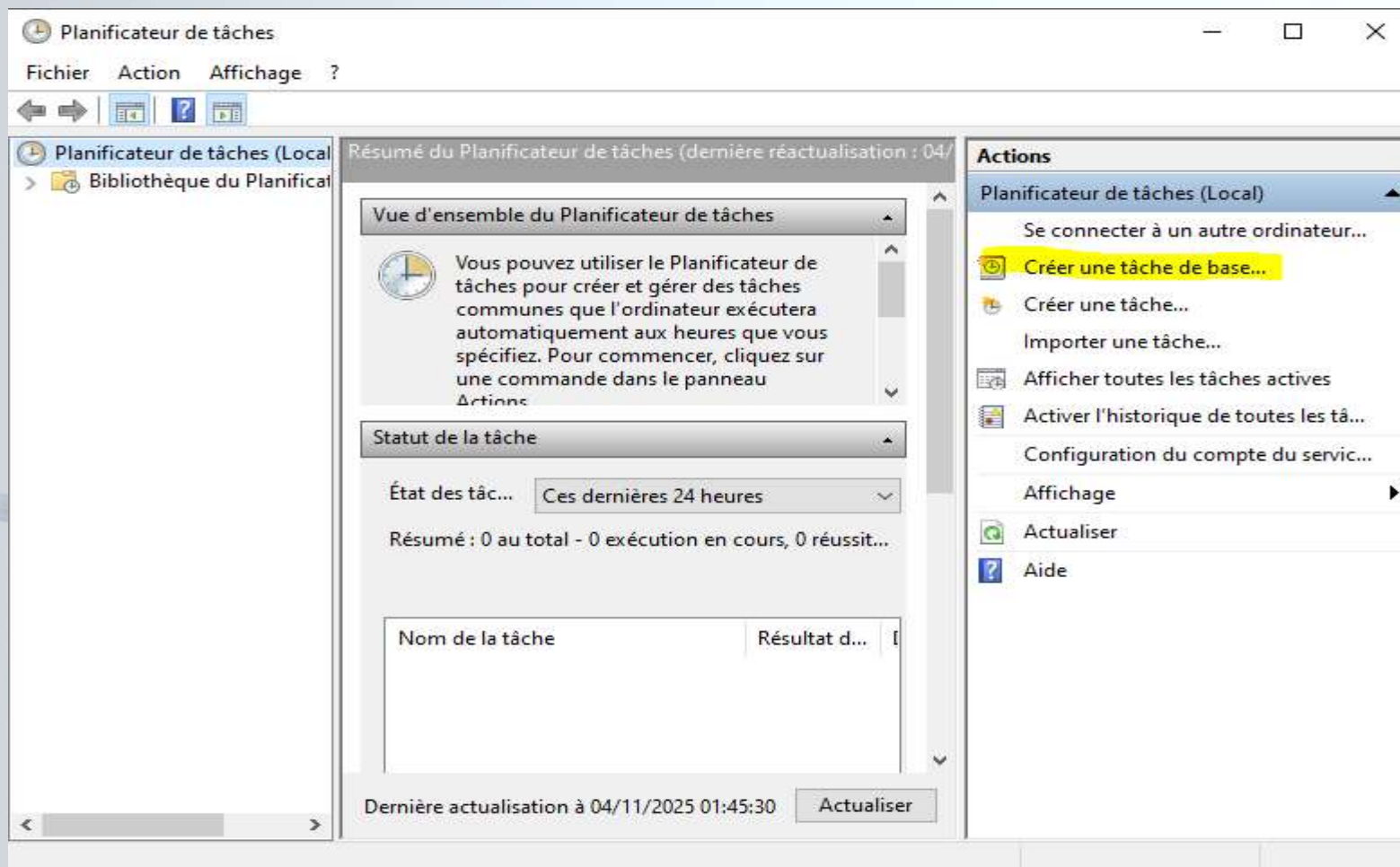
Travail en équipe & contrôle de version (Git / SVN)

- Le contrôle de version permet à plusieurs développeurs de travailler simultanément sur un même projet Talend, tout en :
 - ✓ conservant un historique complet des modifications,
 - ✓ facilitant la collaboration,
 - ✓ évitant les pertes de code
 - ✓ permettant de revenir à une version précédente en cas d'erreur.
 - ✓ ...
- Depuis Talend 7.x et 8.x, Git est le standard.
- Cycle de travail typique avec Git dans Talend
 1. Clone du projet depuis le dépôt Git
 2. Création / modification de jobs, contextes, routines...
 3. Pull des mises à jour des collègues (avec éventuellement des merges)
 4. Commit local des changements
 5. Push vers le dépôt distant
 6. Gestion de branches pour isoler les développements

Industrialisation & Déploiement d'un projet Talend

Déploiement et planification

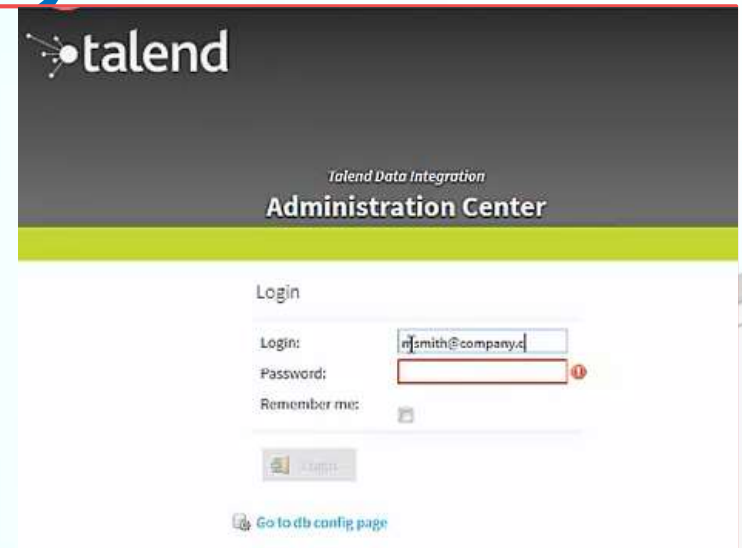
- Planification du Batch : Tache planifié / Cron
- taskschd.msc



Industrialisation & Déploiement d'un projet Talend

Talend Administration Center (TAC) – à savoir

- TAC est une application web (souvent déployée sur Tomcat) :
 - ✓ permet d'administrer, sécuriser, planifier, déployer et superviser les jobs Talend, les utilisateurs et les projets.
 - ✓ Gérer les projets Talend (versions, utilisateurs, autorisations),
 - ✓ Déployer des jobs sur des serveurs distants (JobServers),
 - ✓ planifier des exécutions automatiques (scheduler),
 - ✓ suivre l'historique d'exécution et les logs
- TAC communique avec :
 - ✓ Repository (SVN ou Git) → pour stocker le code source des projets
 - ✓ Database TAC → pour stocker la configuration, les logs, les planifications
 - ✓ JobServer → pour exécuter les jobs déployés
 - ✓ Web Browser → interface utilisateur



Annexe

Génération de la documentation

The screenshot displays the Talend Open Studio interface with a project named 'FormationTalent'. The 'Jobs' tree on the left shows a hierarchy of jobs, including 'tReplaceJob_0.1'. A context menu is open over 'tReplaceJob_0.1', with the option 'Générer la documentation au format HTML' highlighted. The main workspace shows a job diagram for 'tReplaceJob_0.1' with components: 'tRowGenerator_1', 'AllRow', 'row3 (Main)', and 'tReplace_1'. A note indicates: 'Carries out a Search & Replace operation in the input column. Helps to cleanse all files before further processing.' The right-hand pane shows the 'Jobs' tab with details for 'tReplaceJob_0.1', including its description, properties, and a preview of the job diagram. The bottom pane shows the 'Configuration' tab with various settings for the job, including 'Paramètres supplémentaires', 'Statut & Logs', 'Liste des contextes', and 'Liste des composants'. The 'Liste des composants' table lists the components used in the job:

Nom du composant	Type de composant
tRowGenerator_1	Logiciel
AllRow	Logiciel
row3_1	Logiciel
tReplace_1	Logiciel
tRowGenerator_1	Logiciel

The 'Description des composants' section shows the 'tReplace_1' component details, including its name, version, and a list of parameters.

08/11/2025

Atelier 5 : Construire/Exporter un job en JAR exécutable + Scheduling

- Point de départ : Dernier Job résultat de l'atelier 4 (charger la base primaire)
- En se basant/copiant ce Job et en utilisant les composants `tReplace`, `tPreJob`, la routine `Numeric.sequence` effectuez les refactorings suivants :
 - 1) Remplacer les `code_produit` dans le flux qui commence par P0 par A1.
 - 2) Remplacer les `id_produit` dans le flux de tel sorte il y aura pas de chevauchement avec les Ids en base
 - 3) Construire le Job en l'exportant en JAR exécutable
 - 4) Exploiter le fichier zip généré pour créer un exécutable dédié à l'environnement de DEV.
 - 5) En utilisant les tâches planifiées de windows (scheduler) (ou cron pour Linux) , planifier l'exécution du Batch , tous les jours à minuit.

Evaluation



Fin



08/11/2025